Adaptive TDF from any TDF via Pseudorandom Ciphertext PKE

Fuyuki Kitagawa[1] and Takahiro Matsuda[2]

[1] NTT Social Informatics Laboratories, Tokyo, Japan, fuyuki.kitagawa@ntt.com
[2] National Institute of Advanced Industrial Science and Technology (AIST), Tokyo, Japan, t-matsuda@aist.go.jp

Abstract

We present a generic construction of adaptive trapdoor function (TDF) from the combination of any TDF and pseudorandom-ciphertext public-key encryption (PKE) scheme. As a direct corollary, we can obtain adaptive TDF from any trapdoor permutation (TDP) whose domain is both recognizable and sufficiently dense. In our construction, we can prove that the function's output is indistinguishable from uniform even when an adversary has access to the inversion oracle.

Keywords: TDF, adaptive TDF, pseudorandom-ciphertext PKE

Contents

1 Introduction **2**
- 1.1 Backgrounds . . . 2
- 1.2 Our Results . . . 2
- 1.3 Related Work . . . 3
- 1.4 Technical Overview . . . 3
- 1.5 Paper Organization . . . 6

2 Preliminaries **7**
- 2.1 Key Encapsulation Mechanism . . . 7
- 2.2 Trapdoor Function . . . 8
- 2.3 Target Collision Resistant Hash Function . . . 10
- 2.4 Pseudorandom Generator . . . 11
- 2.5 Universal Hash Family and Leftover Hash Lemma . . . 11

3 Randomness-Recoverable KEM with Pseudorandom Ciphertexts **11**
- 3.1 From a TDF with Pseudorandomness . . . 12
- 3.2 From a One-way TDF and a KEM with the Pseudorandom Ciphertext Property 14

4 Our Proposed TDF Construction **16**
- 4.1 Our Construction . . . 16
- 4.2 Proof of Correctness (Proof of Theorem 5) . . . 18
- 4.3 Proof of Security (Proof of Theorem 6) . . . 19

1 Introduction

1.1 Backgrounds

Trapdoor Function. Trapdoor functions (TDFs) [DH76, RSA78] lie at the heart of modern public-key cryptography, predating even public-key encryption (PKE). Informally, a TDF is a one-way function that becomes easy to invert when the secret trapdoor information is given. The notable feature of a TDF is its inversion property. Given the trapdoor, the inversion algorithm of the TDF recovers the entire input, whereas in randomized PKE schemes the decryption algorithm does not necessarily recover the random coins used during encryption. The randomness-recoverability of a TDF is extremely powerful: it implies not only PKE but also the compelling notion of deterministic encryption [FOR12]. Furthermore, the elegant work of Hohenberger, Koppula, and Waters [HKW20] showed how to leverage this property to construct a PKE scheme that is secure against chosen-ciphertext attacks (CCA) [NY90, DDN91, RS92].

Adaptive TDF (ATDF). A TDF is called an *adaptive TDF (ATDF)* [KMO10] if it remains one-way even when the adversary is granted oracle access to the inversion algorithm. Introduced after lossy TDFs [PW08] and correlated-product secure TDFs [RS09], ATDFs were proposed as the weakest known variant of TDFs that are still sufficient to obtain CCA secure PKE. Although Hohenberger et al. [HKW20] later showed that even plain TDFs already imply CCA secure PKE, the notion of ATDFs remains intriguing. For example, if an ATDF additionally ensures that its outputs are computationally indistinguishable from random strings, then its randomness-recoverability property immediately yields CCA secure deterministic encryption schemes.

Kiltz, Mohassel, and O'Neill [KMO10] showed that ATDFs can be obtained from lossy TDFs and correlated-product-secure TDFs. They also provided a concrete ATDF construction under a strengthened variant of the RSA assumption. Kitagawa, Matsuda, and Tanaka [KMT19, KMT22] demonstrated how to build an ATDF by combining a pseudorandom-ciphertext PKE scheme with a secret-key encryption (SKE) scheme that satisfies key-dependent-message (KDM) security [BRS03] and supports randomness-recoverability. Their construction can be instantiated under a broad range of concrete assumptions, but its security is proven only with respect to a somewhat artificial input-sampling algorithm.[1]

In this work, we introduce a new methodology for constructing ATDFs. Especially, we investigate how we can realize ATDFs from any TDFs with the help of mild cryptographic assumptions.

1.2 Our Results

Our main result in this paper is the following.

Theorem 1 (Informal) *Assume that there exist a TDF and a PKE scheme that has pseudorandom ciphertexts. Then, there exists an ATDF.*

Our result is orthogonal to that by Kitagawa et al. [KMT19, KMT22], since there is no known relation between TDFs and KDM secure SKE (with randomness-recoverability). Our result leads to interesting new feasibility results listed below.

[1] In their construction, one of input components is encryption random coins for the KDM-secure SKE scheme, which may follow a non-uniform distribution (e.g., in an LPN-based instantiation of [ACPS09].)

- Both of our building blocks are implied by trapdoor permutations (TDPs) whose domain is both recognizable and sufficiently dense.[2] Thus, we can obtain the following theoretically interesting corollary.

  Corollary 1 (Informal) *Assume that there exists a TDP whose domain is both recognizable and sufficiently dense. Then, there exists an ATDF.*

  Aa far as we know, this is the first formal relationship between TDPs and an extended variant of TDFs.

- Our construction additionally satisfies the property that the function's outputs are computationally indistinguishable from random strings. Such ATDFs immediately imply CCA secure PKE schemes with the pseudorandom ciphertext property, which has many fascinating applications such as selective-opening security [BHY09, FHKW10, LP15], steganography [Hop05, BL18], and cryptographic watermarking [QWZ18]. Our results show that those applications can be based on the combination of any TDF and a PKE scheme with the pseudorandom ciphertext property.

1.3 Related Work

Techniques for constructing TDFs have advanced remarkably in recent years, beginning with the elegant work of Garg and Hajiabadi [GH18]. They achieved the first TDF based on the computational Diffie-Hellman (CDH) assumption. The underlying ideas build on the earlier techniques developed for laconic oblivious transfer [CDG+17] and CDH-based identity-based encryption [DG17].

Subsequently, Garg, Gay, and Hajiabadi [GGH19] extended the techniques of Garg and Hajiabadi by introducing the mirroring technique. Among other contributions, this new approach refined the original construction and yielded the first TDF with (almost-all-keys) perfect correctness under the CDH assumption.

Kitagawa et al. [KMT19, KMT22] proposed a general method for constructing ATDFs based on PKE with the pseudorandom ciphertext property and KDM secure SKE supporting randomness-recoverability. Their design combines the technique from [GGH19] (called the "mirroring" technique) with the beautiful construction technique for CCA secure PKE proposed by Koppula and Waters [KW19].

Garg, Hajiabadi, Malavolta, and Ostrovsky [GHMO21] presented a general method for transforming encryption schemes into their trapdoored (randomness-recoverable) variants, using a hinting PRG [KW19]. Their baseline TDF resembles that of Kitagawa et al. [KMT19, KMT22], except that a hinting PRG is used in place of KDM-secure SKE. They showed how to extend the technique to identity-based and even attribute-based settings.

1.4 Technical Overview

We provide the high level ideas for our construction.

Main Building Block: TDF with Pseudorandom Outputs. Our main building block is a TDF whose outputs are computationally indistinguishable from random strings. Somewhat surprisingly, we show that such a primitive can be constructed from the combination of any TDF and PKE with the pseudorandom ciphertext property. Importantly, *randomness-recoverability is not required for the building-block pseudorandom ciphertext PKE scheme.* The construction of

[2] The precise meaning of "dense" is introduced in Section 3.1.

a TDF with pseudorandom outputs is rather simple. We encrypt the underlying TDF's output by the pseudorandom-ciphertext PKE scheme using the hardcore bits of the TDF as encryption random coins. In this work, we use this primitive with the abstraction of a key encapsulation mechanism (KEM) with randomness-recoverability and the pseudorandom ciphertext property.

TDF with the "Targeted-Lossy" Property from a KEM with Randomness-Recoverability and Pseudorandom Ciphertexts. We first show that we can realize a TDF with "targeted-lossy"-like property [QWW21] based on our building block, using the mirroring technique [GGH19]. We consider the following construction of a TDF TDF based on our building block KEM. For ease of exposition, below we assume the ciphertext space $\mathcal{C}$ and the session-key space $\mathcal{K}$ of KEM are abelian groups and we denote by $+$ the group operations in both $\mathcal{C}$ and $\mathcal{K}$.

We start with the key generation. The construction can be divided into n blocks, which means that each block has its own key, and the evaluation and inversion can be done in block-wise. The evaluation key for the i-th block is of the form $(\mathsf{pk}_i, \mathsf{C}_i, \mathsf{A}_i)$ where pk_i is a public key of KEM and C_i and A_i are random elements sampled from $\mathcal{C}$ and $\mathcal{K}$, respectively. The trapdoor (i.e. inversion key) for the i-th block is sk_i corresponding to pk_i. The input for TDF is of the form $\mathsf{x} := (\mathsf{s}, (\mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]})$, where $\mathsf{s} = (\mathsf{s}_1, \ldots, \mathsf{s}_n) \in \{0,1\}^n$ is an n-bit string and each $\mathsf{r}_i^{\mathsf{s}_i}$ will be interpreted as random coins to execute the encapsulation algorithm $\mathsf{KEM.Encap}$ of KEM. We regard $(\mathsf{s}_i, \mathsf{r}_i^{\mathsf{s}_i})$ as the i-th block of x, where s_i denotes the i-th bit of s. Here, the notation "$\mathsf{r}_i^{\mathsf{s}_i}$" might look strange given that there is no $\mathsf{r}_i^{1-\mathsf{s}_i}$ used, but it will appear in the security proof.

Given an input $\mathsf{x} := (\mathsf{s}, (\mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]})$, the evaluation algorithm $\mathsf{TDF.Eval}$ of TDF evaluates the input's i-th block $(\mathsf{s}_i, \mathsf{r}_i^{\mathsf{s}_i})$ as follows: it executes $(\mathsf{ct}_i^{\mathsf{s}_i}, \mathsf{k}_i^{\mathsf{s}_i}) \leftarrow \mathsf{KEM.Encap}(\mathsf{pk}_i; \mathsf{r}_i^{\mathsf{s}_i})$ and sets

$$(\mathsf{ct}_i, \mathsf{T}_i) := \begin{cases} (\mathsf{ct}_i^0, \mathsf{k}_i^0) & \text{if } \mathsf{s}_i = 0 \\ (\mathsf{ct}_i^1 + \mathsf{C}_i, \mathsf{k}_i^1 + \mathsf{A}_i) & \text{if } \mathsf{s}_i = 1 \end{cases}. \tag{1}$$

The resulting output of $\mathsf{TDF.Eval}$ is $(\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]}$.

The inversion algorithm $\mathsf{TDF.Inv}$ of TDF inverts the i-th block $(\mathsf{ct}_i, \mathsf{T}_i)$ using sk_i, and recovers s_i from $(\mathsf{ct}_i, \mathsf{T}_i)$ by determining which of the two patterns in Eq. (1) the pair $(\mathsf{ct}_i, \mathsf{T}_i)$ satisfies. (It outputs $\perp$ if the pair matches neither pattern.) Since KEM is randomness-recoverable, the process also recovers $\mathsf{r}_i^{\mathsf{s}_i}$ used to generate $(\mathsf{ct}_i^{\mathsf{s}_i}, \mathsf{k}_i^{\mathsf{s}_i})$. By setting the parameters of KEM carefully, the construction satisfies almost-all-keys perfect correctness. In other words, as long as "good" keys $(\mathsf{pk}_i, \mathsf{C}_i, \mathsf{A}_i)$ and sk_i are generated, which occurs with overwhelming probability, we can correctly recover s_i and $\mathsf{r}_i^{\mathsf{s}_i}$ from $(\mathsf{ct}_i, \mathsf{T}_i)$ without any ambiguity, for every $i \in [n]$.

We now analyze the "targeted-lossy"-like property of TDF. More specifically, we can prove that the evaluation key $(\mathsf{pk}_i, \mathsf{C}_i, \mathsf{A}_i)_{i\in[n]}$ can be switched into the "lossy mode" on the target input $\mathsf{x} := (\mathsf{s}, (\mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]})$ so that if we evaluate x with the lossy evaluation key, the output completely loses the information about s. The switching of the key is done by replacing $(\mathsf{C}_i, \mathsf{A}_i)$ in the i-th block evaluation key with $(\mathsf{ct}_i^0 - \mathsf{ct}_i^1, \mathsf{k}_i^0 - \mathsf{k}_i^1)$, where $(\mathsf{ct}_i^v, \mathsf{k}_i^v) \leftarrow \mathsf{KEM.Encap}(\mathsf{pk}_i; r_i^v)$ for both $v \in \{0,1\}$, and $(r_i^{1-\mathsf{s}_i})_{i\in[n]}$ are random strings chosen independently from x, for every $i \in [n]$. We can see that if we evaluate x with this lossy key on x, then the i-th block output is $(\mathsf{ct}_i, \mathsf{T}_i) = (\mathsf{ct}_i^0, \mathsf{k}_i^0)$ regardless of the value of s_i, and the output $(\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]}$ of $\mathsf{TDF.Eval}$ does not have any information about s. The lossy evaluation key is computationally indistinguishable from a normal evaluation key even given the target value x, by the pseudorandom ciphertext property of KEM.

Introducing a Tag into TDF. From here, we start making TDF closer to an adaptive TDF, using the techniques from [KW19, KMT19, KMT22, HKW20]. Firstly, we introduce tags into TDF. We can easily extend the construction of TDF into a tag-based one where $\mathsf{TDF.Eval}$ and $\mathsf{TDF.Inv}$ takes the additional input tag h.

To do so, we add one more random string B into the evaluation key, and the i-th block evaluation on $(\mathsf{s}_i, \mathsf{r}_i^{\mathsf{s}_i})$ with respect to a tag h is done as follows:

$$(\mathsf{ct}_i, \mathsf{T}_i) := \begin{cases} (\mathsf{ct}_i^0, \mathsf{k}_i^0) & \text{if } \mathsf{s}_i = 0 \\ (\mathsf{ct}_i^1 + \mathsf{C}_i, \mathsf{k}_i^1 + \mathsf{A}_i + \mathsf{B} \cdot \mathsf{h}) & \text{if } \mathsf{s}_i = 1 \end{cases}, \tag{2}$$

where we are now assuming that T_i, k_i, A_i, B, and h are elements of a finite field (rather than an abelian group) so that $\mathsf{B} \cdot \mathsf{h}$ is well-defined. The inversion of the i-th block can be done similarly given sk_i and the tag h.

TDF now satisfies the all-but-one targeted-lossy property where the lossy mode is activated only for a designated special tag h^* whereas the injectivity and efficient inversion property are preserved for any other tags. The switching into the all-but-one targeted-lossy mode for $\mathsf{x} := (\mathsf{s}, (\mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]})$ and tag h^* is done by replacing $(\mathsf{C}_i, \mathsf{A}_i)$ in the i-th block evaluation key into $(\mathsf{ct}_i^0 - \mathsf{ct}_i^1, \mathsf{k}_i^0 - \mathsf{k}_i^1 - \mathsf{B} \cdot \mathsf{h}^*)$ instead of $(\mathsf{ct}_i^0 - \mathsf{ct}_i^1, \mathsf{k}_i^0 - \mathsf{k}_i^1)$, where $(\mathsf{ct}_i^v, \mathsf{k}_i^v) \leftarrow \mathsf{KEM.Encap}(\mathsf{pk}_i; r_i^v)$ for both $v \in \{0, 1\}$, and $(r_i^{1-\mathsf{s}_i})_{i\in[n]}$ are fresh random coins as before. We can check that the output of $\mathsf{TDF.Eval}$ on input x and h^* completely loses the information about s. Moreover, we can ensure that the injectivity and the efficient inversion is preserved for any tag other than h^* due to the fact that B is chosen uniformly at random even in the (all-but-one) targeted-lossy mode.

Our Construction ATDF: Injecting Redundancy to TDF. We now show how to extend the above TDF into an ATDF ATDF. We adopt the technique from [HKW20], and inject redundancy into TDF. In this overview, we focus on tag-based ATDF [KMO10] where the adversary can get access to the inversion oracle with respect to any tag that is different from the one used to generate the challenge instance. In the main body, we construct a standard ATDF. The high level idea is as follows.

The evaluation key of ATDF consists of the evaluation key $((\mathsf{pk}_i, \mathsf{C}_i, \mathsf{A}_i)_{i\in[n]}, \mathsf{B})$ of TDF and an additional public key pk_0 of KEM. The inversion key is still set to $(\mathsf{sk}_i)_{i\in[n]}$ of TDF. (sk_0 corresponding to pk_0 is discarded, and used only in the security proof.)

The input to ATDF is the same as TDF thus is of the form $\mathsf{x} := (\mathsf{s}, (\mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]})$. On input x and a tag h, $\mathsf{ATDF.Eval}$ first evaluates it with $\mathsf{TDF.Eval}$ and generates $(\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]}$. Then, we execute KEM with pk_0 using the random coins $\bigoplus_{i\in[n]} \mathsf{r}_i^{\mathsf{s}_i}$, that is, $(\mathsf{ct}_0, \mathsf{k}_0) \leftarrow \mathsf{KEM.Encap}(\mathsf{pk}_0; \bigoplus_{i\in[n]} \mathsf{r}_i^{\mathsf{s}_i})$. The final output of $\mathsf{ATDF.Eval}$ is $\mathsf{y} := (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$. As we already checked above, we can recover $\mathsf{x} := (\mathsf{s}, (\mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]})$ from $(\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]}$ using $(\mathsf{sk}_i)_{i\in[n]}$ and h. After recovering x, $\mathsf{ATDF.Inv}$ checks if ct_0 matches with the ciphertext output by $\mathsf{KEM.Encap}(\mathsf{pk}_0; \bigoplus_{i\in[n]} \mathsf{r}_i^{\mathsf{s}_i})$, and outputs x if and only if the check is passed. Here, k_0 corresponding to ct_0 is not directly used, but it can be used as hardcore bits of an input x if necessary.

Output Pseudorandomness of ATDF. We analyze the output pseudorandomness of ATDF, first without considering the inversion oracle. Let h^* be the challenge tag and $\mathsf{y} = (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$ be the challenge instance that is the output of $\mathsf{ATDF.Eval}$ on input $\mathsf{x} = (\mathsf{s}, (\mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]})$ and tag h^*. We cannot directly apply the ciphertext pseudorandomness of KEM to argue that $\mathsf{y} = (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$ is pseudorandom, since those $n+1$ ciphertexts of KEM are generated using correlated random coins. Nevertheless, we can invoke the all-but-one targeted-lossy property of TDF. This is because switching the evaluation key of TDF into the all-but-one targeted-lossy mode is done using the pseudorandom ciphertext property of KEM with respect to random coins $(\mathsf{r}_i^{1-\mathsf{s}_i})_{i\in[n]}$ chosen independently from $(\mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]}$. Once the evaluation key is switched into the all-but-one targeted-lossy mode on x and h^*, $\mathsf{y} = (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$ loses the information about s. Then, we can change the random coins $\bigoplus_{i\in[n]} \mathsf{r}_i^{\mathsf{s}_i}$ used to generate ct_0 into a uniformly

random string independent of $(\mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]}$ by the leftover hash lemma [HILL99], solving the correlation issue on the random coins for $(\mathsf{ct}_i)_{i\in[n]}$ and ct_0. Then, the output pseudorandomness follows from the pseudorandom ciphertext property of KEM.

Handling the Inversion Oracle. By the additional validity check on ct_0 in the inversion algorithm, ATDF has a property that the inversion can be executed with any n keys out of $n+1$ keys $\mathsf{sk}_0, \mathsf{sk}_1, \ldots, \mathsf{sk}_n$. More specifically, for any $j \in [n]$, consider the following alternative inversion procedure $\mathsf{ATDF.AltInv}_j$ indexed by $j \in [n]$.

- Input: $\mathsf{y} = (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$, $(\mathsf{sk}_0, (\mathsf{sk}_i)_{i\in[n]\setminus\{j\}})$, and h.
- Recover $(\mathsf{s}_i, \mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]\setminus\{j\}}$ from $(\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]\setminus\{j\}}$ using $(\mathsf{sk}_i)_{i\in[n]\setminus\{j\}}$ and h as in TDF.
- Recover r_0 from ct_0 using sk_0.
- Set $\mathsf{r}'_j := \mathsf{r}_0 \oplus (\bigoplus_{i\in[n]\setminus\{j\}} \mathsf{r}_i^{\mathsf{s}_i})$ and execute $(\mathsf{ct}'_j, \mathsf{k}'_j) \leftarrow \mathsf{KEM.Encap}(\mathsf{pk}_j; \mathsf{r}'_j)$.
- Check if $(\mathsf{ct}_j, \mathsf{T}_j)$ matches with either $(\mathsf{ct}'_j, \mathsf{k}'_j)$ or $(\mathsf{ct}'_j + \mathsf{C}_j, \mathsf{k}'_j + \mathsf{A}_j + \mathsf{B}\cdot\mathsf{h})$. In the first case, set $(\mathsf{s}_j, \mathsf{r}_j^{\mathsf{s}_j}) := (0, \mathsf{r}'_j)$, and in the second case, set $(\mathsf{s}_j, \mathsf{r}_j^{\mathsf{s}_j}) := (1, \mathsf{r}'_j)$. If it matches neither pattern, output $\perp$.
- Output $(\mathsf{s}, (\mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]})$.

ATDF satisfies (almost-all-keys) perfect correctness with not only $\mathsf{ATDF.Inv}$ but also any of $\mathsf{ATDF.AltInv}_1, \ldots, \mathsf{ATDF.AltInv}_n$. Moreover, we also assume that given $\mathsf{y} = (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$ and h, all of these inversion processes output the inversion result x' only when they re-execute $\mathsf{ATDF.Eval}$ on x' and h and the result y' matches the given y. Then, we can ensure that all of $\mathsf{ATDF.Inv}, \mathsf{ATDF.AltInv}_1, \ldots, \mathsf{ATDF.AltInv}_n$ are functionally equivalent for any string y: if y is in the image of the $\mathsf{ATDF.Eval}$, they output the corresponding pre-image x from the perfect correctness, and if y is not in the image, they output $\perp$ thanks to the re-evaluation test. Moreover, even if some or even all block evaluation keys of TDF are already switched into all-but-one targeted-lossy mode, the functional equivalence is preserved for any tag other than the one tied to the targeted-lossy mode.

Given the above alternative inversion algorithms, the proof strategy outlined in the previous paragraph still goes through even if we take into account the adversary's access to the inversion oracle. Whenever we use the security of KEM with respect to the key pair $(\mathsf{pk}_i, \mathsf{sk}_i)$ for any $i \in [n]$, we can simulate the inversion oracle with $\mathsf{ATDF.AltInv}_i$.

As stated above, in the main body, we construct a standard ATDF, not a tag-based one. In the construction, we additionally use a target collision resistant hash function, and utilize the hash value of $\mathsf{ct}_0 \| (\mathsf{ct}_i)_{i\in[n]}$ as the tag h in the above construction of ATDF, which is a technique used in some previous works [MH15, KMT19, KMT22]. This achieves a standard ATDF without compromising the output pseudorandomness property of the resulting construction.

1.5 Paper Organization

The rest of the paper is organized as follows. In Section 2, we review the basic notation and cryptographic primitives used in the paper. In Section 3, we show how to build a KEM that simultaneously satisfies the pseudorandom ciphertext property and randomness-recoverability, which is used as the main building block in our TDF construction. In Section 4, we present the proposed ATDF as our main result, and provide its correctness and security proofs.

2 Preliminaries

In this section, we review the basic notation and the definitions of main cryptographic primitives.

Basic Notation. For $n \in \mathbb{N}$, we define $[n] := \{1, \ldots, n\}$. For strings x and y, "$|x|$" denotes the bit-length of x, and "$(x \stackrel{?}{=} y)$" is the operation that returns 1 if $x = y$ and 0 otherwise. For a discrete finite set S, "$|S|$" denotes its size, and "$x \xleftarrow{r} S$" denotes choosing an element x uniformly at random from S. For a (probabilistic) algorithm A, "$y \leftarrow \mathsf{A}(x)$" denotes assigning to y the output of $\mathcal{A}$ on input x, and if we need to specify a randomness r used in A, we write "$y \leftarrow \mathsf{A}(x; r)$". If furthermore $\mathcal{O}$ is a function or an algorithm, then "$\mathsf{A}^{\mathcal{O}}$" means that A has oracle access to $\mathcal{O}$. A function $\epsilon(\lambda) : \mathbb{N} \to [0, 1]$ is said to be *negligible* if $\epsilon(\lambda) = \lambda^{-\omega(1)}$. We write $\epsilon(\lambda) = \mathsf{negl}(\lambda)$ to mean ϵ being negligible. "PPT" stands for *probabilistic polynomial time*.

2.1 Key Encapsulation Mechanism

In this paper, we will use a key encapsulation mechanism (KEM) that satisfies randomness-recoverability and the security notion called the pseudorandom ciphertext property.[3] Furthermore, for the definition of correctness, we formalize "almost-all-keys" correctness, which is naturally adapted from the definition for PKE formalized by Dwork, Naor, and Reingold [DNR04]. Hence, we review the necessary definitions.

Definition 1 (Key Encapsulation Mechanism) *A key encapsulation mechanism (KEM)* KEM *consists of the three PPT algorithms* $(\mathsf{KKG}, \mathsf{Encap}, \mathsf{Decap})$*:*

- KKG *is the key generation algorithm that takes* 1^λ *as input, and outputs a public/secret key pair* $(\mathsf{pk}, \mathsf{sk})$*. We assume that the security parameter* λ *determines the ciphertext space* $\mathcal{C}$*, the session-key space* $\mathcal{K}$*, and the randomness space* $\mathcal{R}$ *of* Encap*.*
- Encap *is the encapsulation algorithm that takes a public key* pk *as input, and outputs a ciphertext/session-key pair* $(\mathsf{ct}, \mathsf{k})$*.*
- Decap *is the (deterministic) decapsulation algorithm that takes a secret key* sk *and a ciphertext* ct *as input, and outputs a session-key* k *or the invalidity symbol* $\perp \notin \mathcal{K}$*.*

Let $\epsilon : \mathbb{N} \to [0, 1]$*. We say that a KEM* $\mathsf{KEM} = (\mathsf{KKG}, \mathsf{Encap}, \mathsf{Decap})$ *is* ϵ-almost-all-keys correct *if we have*

$$\Pr_{(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{KKG}(1^\lambda)} \left[\begin{array}{ll} \exists \mathsf{r} \in \mathcal{R} \text{ s.t.} & \mathsf{Encap}(\mathsf{pk}; \mathsf{r}) = (\mathsf{ct}, \mathsf{k}) \\ & \wedge\ \mathsf{Decap}(\mathsf{sk}, \mathsf{ct}) \neq \mathsf{k} \end{array} \right] \leq \epsilon(\lambda).$$

(A public key pk *under which incorrect decapsulation could occur is called* erroneous*.) Furthermore, we just say that* KEM *is* correct *(resp.* almost-all-keys correct*) if it is* 0*-almost-all-keys correct (resp.* negl*-almost-all-keys correct).*

Randomness-Recoverability. In our proposed construction of a TDF, we will use a KEM that satisfies *randomness-recoverability*, which requires that for an honestly generated ciphertext, the randomness used to generate it can be recovered in the decapsulation process. We formally define the property as follows.

[3] Note that in terms of existence, a KEM with randomness-recoverability and the pseudorandom ciphertext property is equivalent to a PKE scheme satisfying the same properties.

$\mathsf{Expt}^{\mathsf{prct}}_{\mathsf{KEM},\mathcal{A}}(\lambda)$:
$(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KKG}(1^\lambda)$
$(\mathsf{ct}^*_1, \mathsf{k}^*_1) \leftarrow \mathsf{Encap}(\mathsf{pk})$
$(\mathsf{ct}^*_0, \mathsf{k}^*_0) \xleftarrow{\mathsf{r}} \mathcal{C} \times \mathcal{K}$
$b \xleftarrow{\mathsf{r}} \{0,1\}$
$b' \leftarrow \mathcal{A}(\mathsf{pk}, \mathsf{ct}^*_b, \mathsf{k}^*_b)$
Return $(b' \stackrel{?}{=} b)$.

Figure 1: The pseudorandom ciphertext experiment for a KEM.

Definition 2 (Randomness-Recoverable KEM) *Let* $\mathsf{KEM} = (\mathsf{KKG}, \mathsf{Encap}, \mathsf{Decap})$ *be a KEM such that the randomness space of* Encap *is* $\{0,1\}^\ell$ *for some polynomial* $\ell = \ell(\lambda)$*. We say that* KEM *is* randomness-recoverable *if there exists a deterministic PT algorithm* RDecap *(called the* randomness-recovering decapsulation *algorithm) that takes a secret key* sk *and a ciphertext* ct *as input, and outputs a pair of session-key and randomness (supposedly used in* Encap*)* $(\mathsf{k}, \mathsf{r}) \in \mathcal{K} \times \mathcal{R}$ *or* $\perp$*.*

Let $\epsilon : \mathbb{N} \to [0,1]$*. We say that a randomness-recoverable KEM* KEM *is* ϵ-almost-all-keys correct *if we have*

$$\Pr_{(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{KKG}(1^\lambda)} \left[\exists (\mathsf{ct}, \mathsf{r}) \in \mathcal{C} \times \mathcal{R} \text{ s.t. } \begin{array}{l} \mathsf{RDecap}(\mathsf{sk}, \mathsf{ct}) = (*, \mathsf{r}) \\ \wedge\ \mathsf{Encap}(\mathsf{pk}; \mathsf{r}) \neq (\mathsf{ct}, *) \end{array} \right] \leq \epsilon(\lambda).$$

(A public key pk *under which incorrect randomness-recovering decapsulation could occur is called* erroneous*.) Furthermore, we just say that a randomness-recoverable* KEM *is* correct *(resp.* almost-all-keys correct*) if it is* 0*-almost-all-keys correct (resp.* negl*-almost-all-keys correct).*

Pseudorandom Ciphertext Property. Here, we review the security definition for a KEM called the pseudorandom ciphertext property (sometimes also called IND$-CPA security), which requires that a ciphertext is pseudorandom in the ciphertext space, and thus is a stronger notion than the standard IND-CPA security for a KEM.

Definition 3 (Pseudorandom Ciphertext Property for a KEM) *Let* $\mathsf{KEM} = (\mathsf{KKG}, \mathsf{Encap}, \mathsf{Decap})$ *be a KEM whose ciphertext and session-key spaces are* $\mathcal{C}$ *and* $\mathcal{K}$*, respectively. We say that* KEM *satisfies the* pseudorandom ciphertext *property if for all PPT adversaries* $\mathcal{A}$*, we have* $\mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM},\mathcal{A}}(\lambda) := 2 \cdot |\Pr[\mathsf{Expt}^{\mathsf{prct}}_{\mathsf{KEM},\mathcal{A}}(\lambda) = 1] - 1/2| = \mathsf{negl}(\lambda)$*, where* $\mathsf{Expt}^{\mathsf{prct}}_{\mathsf{KEM},\mathcal{A}}(\lambda)$ *is defined as in Figure 1.*

2.2 Trapdoor Function

Here, we review the definitions for a TDF. As in the KEM case, for correctness, we will define almost-all-keys correctness.

Definition 4 (Trapdoor Function) *A trapdoor function (TDF)* TDF *consists of the four PPT algorithms* $(\mathsf{Setup}, \mathsf{Samp}, \mathsf{Eval}, \mathsf{Inv})$*:*

- Setup *is the setup algorithm that takes* 1^λ *as input, and outputs an evaluation key/trapdoor pair* $(\mathsf{ek}, \mathsf{td})$*. We assume that the security parameter* λ *determines the domain* $\mathcal{X}$ *and the range* $\mathcal{Y}$*.*
- Samp *is the domain sampling algorithm that takes* 1^λ *as input, and outputs a domain element* $x \in \mathcal{X}$*.*

- Eval *is the evaluation algorithm that takes an evaluation key* ek *and a domain element* x *as input, and outputs some element* $y \in \mathcal{Y}$.
- Inv *is the (deterministic) inversion algorithm that takes a trapdoor* td *and an element* $y \in \mathcal{Y}$ *as input, and outputs some element* $x \in \mathcal{X} \cup \{\perp\}$ *which could be the invalidity symbol* $\perp \notin \mathcal{X}$.

Let $\epsilon : \mathbb{N} \to [0,1]$. *We say that a TDF* $\mathsf{TDF} = (\mathsf{Setup}, \mathsf{Samp}, \mathsf{Eval}, \mathsf{Inv})$ *is* ϵ-almost-all-keys correct *if we have*

$$\Pr_{(\mathsf{ek},\mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda)} \left[\exists x \in \mathcal{X} \text{ s.t. } \mathsf{Inv}(\mathsf{td}, \mathsf{Eval}(\mathsf{ek}, x)) \neq x \right] \leq \epsilon(\lambda).$$

(An evaluation key ek *under which incorrect inversion could occur is called* erroneous.[4]*) Furthermore, we just say that* TDF *is* correct *(resp.* almost-all-keys correct*) if it is 0-almost-all-keys correct (resp.* negl*-almost-all-keys-correct).*

Security Notions for a TDF. For security notions for a TDF, we consider *adaptive one-wayness* (one-wayness in the presence of the inversion oracle) [KMO10], and *adaptive pseudorandomness* (pseudorandomness of an evaluation result in the presence of the inversion oracle).

Definition 5 (Security Notions for a TDF) *Let* $\mathsf{TDF} = (\mathsf{Setup}, \mathsf{Samp}, \mathsf{Eval}, \mathsf{Inv})$ *be a TDF. We say that* TDF *satisfies*

- adaptive one-wayness *if for all PPT adversaries* $\mathcal{A}$*, we have* $\mathsf{Adv}^{\mathsf{aow}}_{\mathsf{TDF},\mathcal{A}}(\lambda) := \Pr[\mathsf{Expt}^{\mathsf{aow}}_{\mathsf{TDF},\mathcal{A}}(\lambda) = 1] = \mathsf{negl}(\lambda)$*, where* $\mathsf{Expt}^{\mathsf{aow}}_{\mathsf{TDF},\mathcal{A}}(\lambda)$ *is defined as in Figure 2 (left), and in the experiment,* $\mathcal{A}$ *is not allowed to submit* y^* *to the inversion oracle* $\mathsf{Inv}(\mathsf{td}, \cdot)$.
- one-wayness *if for all PPT adversaries* $\mathcal{A}$*, we have* $\mathsf{Adv}^{\mathsf{ow}}_{\mathsf{TDF},\mathcal{A}}(\lambda) := \Pr[\mathsf{Expt}^{\mathsf{ow}}_{\mathsf{TDF},\mathcal{A}}(\lambda) = 1] = \mathsf{negl}(\lambda)$*, where* $\mathsf{Expt}^{\mathsf{ow}}_{\mathsf{TDF},\mathcal{A}}(\lambda)$ *is defined in exactly the same way as* $\mathsf{Expt}^{\mathsf{aow}}_{\mathsf{TDF},\mathcal{A}}(\lambda)$*, except that* $\mathcal{A}$ *is not allowed to use the inversion oracle* $\mathsf{Inv}(\mathsf{td}, \cdot)$.
- adaptive pseudorandomness *if for all PPT adversaries* $\mathcal{A}$*, we have* $\mathsf{Adv}^{\mathsf{apr}}_{\mathsf{TDF},\mathcal{A}}(\lambda) := 2 \cdot |\Pr[\mathsf{Expt}^{\mathsf{apr}}_{\mathsf{TDF},\mathcal{A}}(\lambda) = 1] - 1/2| = \mathsf{negl}(\lambda)$*, where* $\mathsf{Expt}^{\mathsf{apr}}_{\mathsf{TDF},\mathcal{A}}(\lambda)$ *is defined as in Figure 2 (center), and in the experiment,* $\mathcal{A}$ *is not allowed to submit* y^*_b *to the inversion oracle* $\mathsf{Inv}(\mathsf{td}, \cdot)$.
- pseudorandomness *if for all PPT adversaries* $\mathcal{A}$*, we have* $\mathsf{Adv}^{\mathsf{pr}}_{\mathsf{TDF},\mathcal{A}}(\lambda) := 2 \cdot |\Pr[\mathsf{Expt}^{\mathsf{pr}}_{\mathsf{TDF},\mathcal{A}}(\lambda) = 1] - 1/2| = \mathsf{negl}(\lambda)$*, where* $\mathsf{Expt}^{\mathsf{pr}}_{\mathsf{TDF},\mathcal{A}}(\lambda)$ *is defined in exactly the same way as* $\mathsf{Expt}^{\mathsf{apr}}_{\mathsf{TDF},\mathcal{A}}(\lambda)$*, except that* $\mathcal{A}$ *is not allowed to use the inversion oracle* $\mathsf{Inv}(\mathsf{td}, \cdot)$.

It is straightforward to see that the following implication generically holds.

Theorem 2 *Let* TDF *be a TDF with domain* $\mathcal{X}$ *and range* $\mathcal{Y}$*. Assume that* TDF *satisfies adaptive pseudorandomness (resp. pseudorandomness), and the ratio* $|\mathcal{X}|/|\mathcal{Y}|$ *of the sizes of the domain and range is negligible in* λ*. Then,* TDF *also satisfies adaptive one-wayness (resp. one-wayness).*

[4] Note that if ek is not erroneous, then $\mathsf{Eval}(\mathsf{ek}, \cdot)$ is injective.

$\mathsf{Expt}^{\mathsf{aow}}_{\mathsf{TDF},\mathcal{A}}(\lambda)$:
$(\mathsf{ek}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda)$
$\mathsf{x}^* \leftarrow \mathsf{Samp}(1^\lambda)$
$\mathsf{y}^* \leftarrow \mathsf{Eval}(\mathsf{ek}, \mathsf{x}^*)$
$\mathsf{x}' \leftarrow \mathcal{A}^{\mathsf{Inv}(\mathsf{td},\cdot)}(\mathsf{ek}, \mathsf{y}^*)$
Return $(\mathsf{Eval}(\mathsf{ek}, \mathsf{x}') \stackrel{?}{=} \mathsf{y}^*)$.

$\mathsf{Expt}^{\mathsf{apr}}_{\mathsf{TDF},\mathcal{A}}(\lambda)$:
$(\mathsf{ek}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda)$
$\mathsf{x}^* \leftarrow \mathsf{Samp}(1^\lambda)$
$\mathsf{y}^*_1 \leftarrow \mathsf{Eval}(\mathsf{ek}, \mathsf{x}^*)$
$\mathsf{y}^*_0 \xleftarrow{\mathsf{r}} \mathcal{Y}$
$b \xleftarrow{\mathsf{r}} \{0,1\}$
$b' \leftarrow \mathcal{A}^{\mathsf{Inv}(\mathsf{td},\cdot)}(\mathsf{ek}, \mathsf{y}^*_b)$
Return $(b' \stackrel{?}{=} b)$.

$\mathsf{Expt}^{\mathsf{hc}}_{\mathsf{TDF},\mathsf{hc},\mathcal{A}}(\lambda)$:
$(\mathsf{ek}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda)$
$\mathsf{x}^* \leftarrow \mathsf{Samp}(1^\lambda)$
$\mathsf{y}^* \leftarrow \mathsf{Eval}(\mathsf{ek}, \mathsf{x}^*)$
$\mathsf{s}_1 \leftarrow \mathsf{hc}(\mathsf{ek}, \mathsf{x}^*)$
$\mathsf{s}_0 \xleftarrow{\mathsf{r}} \{0,1\}^{|\mathsf{s}_1|}$
$b \xleftarrow{\mathsf{r}} \{0,1\}$
$b' \leftarrow \mathcal{A}(\mathsf{ek}, \mathsf{y}^*, \mathsf{s}_b)$
Return $(b' \stackrel{?}{=} b)$.

Figure 2: Security experiments for a TDF: The adaptive one-wayness experiment (left), the adaptive pseudorandomness experiment (center), and the security experiment for a hardcore function (right). In the adaptive one-wayness (resp. adaptive pseudorandomness) experiment, $\mathcal{A}$ is not allowed to submit the challenge instance y^* (resp. y^*_b) to the inversion oracle $\mathsf{Inv}(\mathsf{td}, \cdot)$.

Proof of Theorem 2. Since the implication in the ordinary (non-adaptive) setting is analogous to the adaptive case, we only show the proof for the latter. Let $\mathcal{A}$ be any adversary against the adaptive one-wayness of TDF. Consider another adversary $\mathcal{B}$ attacking the adaptive pseudorandomness of TDF: Given ek and the challenge instance y^*_b for adaptive pseudorandomness, $\mathcal{B}$ gives $(\mathsf{ek}, \mathsf{y}^*_b)$ to $\mathcal{A}$, whose inversion queries are answered using that of $\mathcal{B}$. When $\mathcal{A}$ terminates with output a candidate inversion result x', $\mathcal{B}$ outputs 1 if and only if $\mathsf{Eval}(\mathsf{ek}, \mathsf{x}') = \mathsf{y}^*_b$ holds. If $b = 1$, the probability that $\mathcal{B}$ outputs 1 is exactly $\mathsf{Adv}^{\mathsf{aow}}_{\mathsf{TDF},\mathcal{A}}(\lambda)$. On the other hand, if $b = 0$, the probability that the challenge instance y^*_0 belongs to the image of $\mathsf{Eval}(\mathsf{ek}, \cdot)$ is at most $|\mathcal{X}|/|\mathcal{Y}| = \mathsf{negl}(\lambda)$, and thus $\mathcal{A}$ can succeed in inverting y^*_0 (and consequently $\mathcal{B}$ outputs 1) only with negligible probability. Hence, we have $\mathsf{Adv}^{\mathsf{apr}}_{\mathsf{TDF},\mathcal{B}}(\lambda) \geq \mathsf{Adv}^{\mathsf{aow}}_{\mathsf{TDF},\mathcal{A}}(\lambda) - \mathsf{negl}(\lambda)$, which completes the proof. □ (**Theorem 2**)

Hardcore Function for a TDF. Here, we recall the notion of a hardcore function for a TDF.

Definition 6 (Hardcore Function for a TDF) *Let* $\mathsf{TDF} = (\mathsf{Setup}, \mathsf{Samp}, \mathsf{Eval}, \mathsf{Inv})$ *be a TDF with domain* $\mathcal{X}$*. Let* hc *be an efficiently computable function that takes an evaluation key* ek *and a domain element* $\mathsf{x} \in \mathcal{X}$ *as input, and outputs a bit-string. We say that* hc *is a* hardcore function *for* TDF *if for all PPT adversaries* $\mathcal{A}$*, we have* $\mathsf{Adv}^{\mathsf{hc}}_{\mathsf{TDF},\mathsf{hc},\mathcal{A}}(\lambda) := 2 \cdot |\Pr[\mathsf{Expt}^{\mathsf{hc}}_{\mathsf{TDF},\mathsf{hc},\mathcal{A}}(\lambda) = 1] - 1/2| = \mathsf{negl}(\lambda)$*, where the experiment* $\mathsf{Expt}^{\mathsf{hc}}_{\mathsf{TDF},\mathsf{hc},\mathcal{A}}(\lambda)$ *is defined as in Figure 2 (right).*

Due to Goldreich and Levin [GL89], and simple parallelization of inputs, we can turn any TDF with one-wayness into one that has a hardcore function whose output length is an arbitrary polynomial in the security parameter λ.

We remark that a TDF with adaptive pseudorandomness and a hardcore function immediately implies the existence of a PKE scheme and a KEM that satisfy the pseudorandom ciphertext property under chosen-ciphertext attacks (i.e. even in the presence of the decryption/decapsulation oracle).

2.3 Target Collision Resistant Hash Function

Definition 7 (Target Collision Resistant Hash Function) *A keyed hash function* Hash *consists of two algorithms* $(\mathsf{HKG}, \mathsf{H})$*:* HKG *is the key generation algorithm that takes* 1^λ *as input, and outputs a hash key* hk*;* H *is the hash evaluation algorithm that takes* hk *and an element* $x \in \{0,1\}^*$ *as input, and outputs a hash value* $y \in \{0,1\}^\lambda$*.*

Hash *is said to be* target collision resistant *if for all PPT adversaries* $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$*, we have*

$$\mathsf{Adv}^{\mathsf{tcr}}_{\mathsf{Hash},\mathcal{A}}(\lambda) := \Pr\left[\begin{array}{l}(x,\mathsf{st}) \leftarrow \mathcal{A}_1(1^\lambda);\ \mathsf{hk} \leftarrow \mathsf{HKG}(1^\lambda);\ x' \leftarrow \mathcal{A}_2(\mathsf{hk},\mathsf{st}) : \\ \mathsf{H}(\mathsf{hk},x') = \mathsf{H}(\mathsf{hk},x) \wedge x' \neq x\end{array}\right] = \mathsf{negl}(\lambda).$$

A target collision resistant hash function can be constructed from any one-way function [NY89, Rom90].

2.4 Pseudorandom Generator

Definition 8 (Pseudorandom Generator) *Let* $\mathsf{G} : \{0,1\}^* \to \{0,1\}^*$ *be an efficiently computable function such that there exists a polynomial* $\ell = \ell(\lambda) > \lambda$ *such that for all* $\mathsf{x} \in \{0,1\}^\lambda$*, we have* $|\mathsf{G}(\mathsf{x})| = \ell(|\mathsf{x}|)$*.* G *is said to be a* pseudorandom generator *(PRG) if for all PPT adversaries* $\mathcal{A}$*, we have*

$$\mathsf{Adv}^{\mathsf{prg}}_{\mathsf{G},\mathcal{A}}(\lambda) := \left|\Pr_{\mathsf{x}\xleftarrow{\mathsf{r}}\{0,1\}^\lambda}[\mathcal{A}(1^\lambda,\mathsf{G}(\mathsf{x}))=1] - \Pr_{\mathsf{y}\xleftarrow{\mathsf{r}}\{0,1\}^\ell}[\mathcal{A}(1^\lambda,\mathsf{y})=1]\right| = \mathsf{negl}(\lambda).$$

A PRG can be constructed from any one-way function [HILL99].

2.5 Universal Hash Family and Leftover Hash Lemma

Let $\mathcal{D}$ and $\mathcal{R}$ be finite sets, and let $\mathcal{H} = \{H : \mathcal{D} \to \mathcal{R}\}$ be a family of hash functions. $\mathcal{H}$ is said to be a *universal hash family*, if for all distinct $\mathsf{x}, \mathsf{x}' \in \mathcal{D}$, we have $\Pr_{H\leftarrow\mathcal{H}}[H(\mathsf{x}) = H(\mathsf{x}')] \leq |\mathcal{R}|^{-1}$.

In this paper, we will use the leftover hash lemma [HILL99] of the following form.

Lemma 1 (Leftover Hash Lemma [HILL99]) *Let* $n, \ell \in \mathbb{N}$*, and let* $\mathcal{H} = \{H : \{0,1\}^n \to \{0,1\}^\ell\}$ *be a universal hash family. Then, for all (computationally unbounded) adversaries* $\mathcal{A}$*, we have*

$$\left|\Pr[\mathcal{A}(H, H(\mathsf{s})) = 1] - \Pr[\mathcal{A}(H,\mathsf{r}) = 1]\right| \leq \frac{1}{2}\sqrt{2^{\ell-n}},$$

where $H \xleftarrow{\mathsf{r}} \mathcal{H}$*,* $\mathsf{s} \xleftarrow{\mathsf{r}} \{0,1\}^n$*, and* $\mathsf{r} \xleftarrow{\mathsf{r}} \{0,1\}^\ell$*.*

3 Randomness-Recoverable KEM with Pseudorandom Ciphertexts

In our proposed TDF construction presented in Section 4, as the main building block, we will use a randomness-recoverable KEM $\mathsf{KEM} = (\mathsf{KKG}, \mathsf{Encap}, \mathsf{Decap}, \mathsf{RDecap})$ that satisfies the pseudorandom ciphertext property and the following additional structural properties (when the security parameter is λ):

- (1) The session key space is $\{0,1\}^{4\lambda}$.
- (2) The randomness space of Encap is $\{0,1\}^{\ell_\mathsf{r}}$ for some polynomial $\ell_\mathsf{r} = \ell_\mathsf{r}(\lambda)$.
- (3) The ciphertext space $\mathcal{C}$ forms an abelian group (where we use the additive notation) and satisfies $|\mathcal{C}| \geq 2^{\ell_\mathsf{r}+\lambda}$.
- (4) The number of possible session-keys k that could be output by $\mathsf{Encap}(\mathsf{pk};\cdot)$ for any pk output by $\mathsf{KKG}(1^\lambda)$, is at most 2^λ.

In this section, we show how to construct such a randomness-recoverable KEM from other primitives. Specifically, in Section 3.1, we show a construction from a TDF with pseudorandomness and a PRG, and in Section 3.2, we show a construction from the combination of a one-way TDF and a (non-randomness-recoverable) KEM with the pseudorandom ciphertext property.

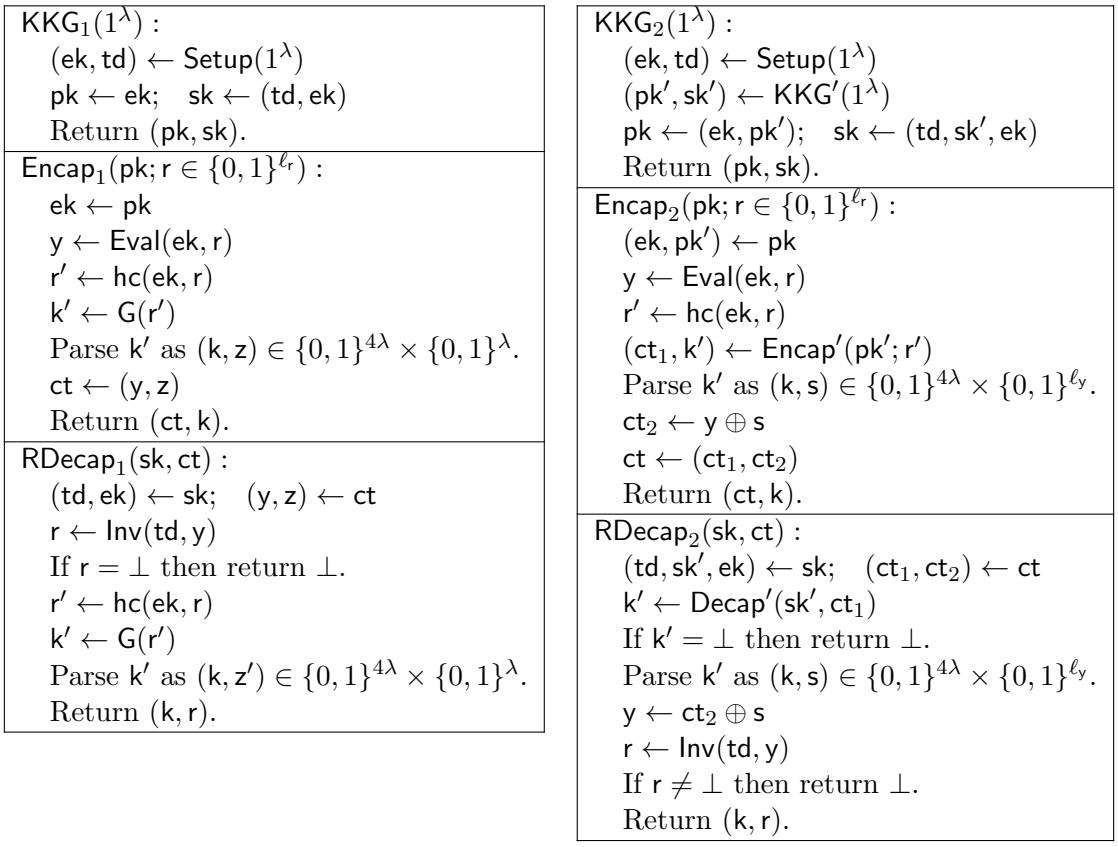

Figure 3: The constructions of a randomness-recoverable KEM: KEM_1 based on a TDF with pseudorandomness (left), and KEM_2 based on a TDF with one-wayness and a KEM with the pseudorandom ciphertext property (right).

3.1 From a TDF with Pseudorandomness

Let $\mathsf{TDF} = (\mathsf{Setup}, \mathsf{Samp}, \mathsf{Eval}, \mathsf{Inv})$ be a TDF such that (when the security parameter is λ) (1) its domain is $\{0,1\}^{\ell_\mathsf{r}}$ for some polynomial $\ell_\mathsf{r} = \ell_\mathsf{r}(\lambda)$, (2) Samp is a uniform distribution over $\{0,1\}^{\ell_\mathsf{r}}$, (3) its range $\mathcal{Y}$ forms an abelian group (where we use additive notation), (4) it satisfies pseudorandomness, and (5) it has a hardcore function hc whose output length is λ. Let $\mathsf{G} : \{0,1\}^\lambda \to \{0,1\}^{5\lambda}$ be a PRG.

Using TDF and G, we construct a randomness-recoverable KEM $\mathsf{KEM}_1 = (\mathsf{KKG}_1, \mathsf{Encap}_1, \mathsf{RDecap}_1)$ as in Figure 3 (left).[5]

It is straightforward to see that KEM_1 is randomness-recoverable, and satisfies the pseudorandom ciphertext property based on the pseudorandomness of the underlying TDF, and the security of the hardcore function hc and the PRG G. For completeness, we provide its formal proof at the end of Section 3.1.

Theorem 3 *If* TDF *is pseudorandom,* hc *is a hardcore function for* TDF*, and* G *is a secure PRG, then* KEM_1 *satisfies the pseudorandom ciphertext property.*

Furthermore, it is immediate to see that KEM_1 satisfies the properties (1), (2), and (4) listed at the beginning of this section. The property (3) is also satisfied since the ciphertext space $\mathcal{C}$

[5]Since the ordinary decapsulation algorithm is redundant for a randomness-recoverable KEM, we omit it in the figure.

of KEM_1 is $\mathcal{Y} \times \{0,1\}^\lambda$ and thus forms an abelian group where for $\mathsf{ct} = (\mathsf{y}, \mathsf{z})$ and $\mathsf{ct}' = (\mathsf{y}', \mathsf{z}')$, we can define $\mathsf{ct} + \mathsf{ct}'$ by $(\mathsf{y} + \mathsf{y}', \mathsf{z} \oplus \mathsf{z}')$.[6] Besides, since $|\mathcal{Y}| \geq 2^{\ell_\mathsf{r}}$, we have $|\mathcal{C}| = |\mathcal{Y}| \cdot 2^\lambda \geq 2^{\ell_\mathsf{r}+\lambda}$, as desired.[7]

Construction from a Trapdoor Permutation with the Canonical Domain Sampling Property. It is also possible to construct a randomness-recoverable KEM from a trapdoor permutation (TDP) that has the property called *canonical domain sampling* [GLN11].

Recall that a TDF with domain $\mathcal{X}$ is said to be a *trapdoor permutation* if $\mathsf{Eval}(\mathsf{ek}, \cdot)$ is a permutation over its domain $\mathcal{X}$ (and thus $\mathsf{Inv}(\mathsf{td}, \cdot)$ is the corresponding inverse permutation over $\mathcal{X}$).[8] A TDP is said to have the *canonical domain sampling* property [GLN11] if (1) the membership of the domain $\mathcal{X}$ can be efficiently checked, and (2) the domain $\mathcal{X}$ is sufficiently "dense", in the sense that there exist a polynomial-time computable function $\ell = \ell(\lambda)$ and a polynomial $p = p(\lambda)$ such that $\mathcal{X} \subseteq \{0,1\}^\ell$ and $|\mathcal{X}| \geq 2^\ell / p$.

If a TDP satisfies the canonical domain sampling property, then it can be converted into one whose domain is $\{0,1\}^\ell$, as shown in [GLN11]: Specifically, given a string $\mathsf{x}' \in \{0,1\}^\ell$, the evaluation algorithm checks if $\mathsf{x}' \in \mathcal{X}$, and runs the original evaluation algorithm if this is the case, and otherwise (i.e. $\mathsf{x}' \notin \mathcal{X}$), it just outputs x' as it is. This converted version of a TDP clearly preserves the permutation property (now its domain is $\{0,1\}^\ell$), and it also satisfies weak one-wayness (where the advantage of an adversary may not be negligible but is bounded by $1 - 1/q(\lambda)$ for some fixed polynomial $q = q(\lambda)$). Then, from a TDP with weak one-wayness, one can construct a TDP with ordinary one-wayness by applying an appropriate hardness amplification method (see, e.g. [Gol01]), while preserving the property that its domain is the set of bit-strings.

Note that if the domain of a one-way TDP is the set of bit-strings, then the TDP unconditionally satisfies pseudorandomness perfectly, and thus from the above construction KEM_1, we can construct a randomness-recoverable KEM with the desired properties.

We now provide the formal proof of Theorem 3.

Proof of Theorem 3. Let $\mathcal{A}$ be any PPT adversary that attacks the pseudorandom ciphertext property of KEM_1. We show that for this $\mathcal{A}$, there exist PPT adversaries $\mathcal{B}$, $\mathcal{B}'$, and $\mathcal{B}''$ that satisfy

$$\mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM}_1,\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\mathsf{hc}}_{\mathsf{TDF},\mathsf{hc},\mathcal{B}}(\lambda) + \mathsf{Adv}^{\mathsf{prg}}_{\mathsf{G},\mathcal{B}'}(\lambda) + \mathsf{Adv}^{\mathsf{pr}}_{\mathsf{TDF},\mathcal{B}''}(\lambda),$$

which implies the theorem.

Our proof is via a sequence of games argument using four games. In the following, let $\mathtt{T}_t$ denote the event that $\mathcal{A}$ outputs $b' = 1$ in Game $t \in [4]$.

Game 1: This is the pseudorandom ciphertext experiment $\mathsf{Expt}^{\mathsf{prct}}_{\mathsf{KEM}_1,\mathcal{A}}(\lambda)$ in which $b = 1$.

Specifically, the description of Game 1 is as follows:

- Compute $(\mathsf{ek}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda)$ and set $\mathsf{pk} \leftarrow \mathsf{ek}$.
- Generate $\mathsf{ct}^* = (\mathsf{y}^*, \mathsf{z}^*)$ and k^* as follows.
  1. Pick $\mathsf{r}^* \xleftarrow{\mathsf{r}} \{0,1\}^{\ell_\mathsf{r}}$ uniformly at random.
  2. Compute $\mathsf{y}^* \leftarrow \mathsf{Eval}(\mathsf{ek}, \mathsf{r}^*)$.

[6] Here, the addition for the first components is the group operation of $\mathcal{Y}$.

[7] The role of the element z included in ct is to guarantee this property on the size of the ciphertext space. If the underlying TDF by itself satisfies $|\mathcal{Y}| \geq 2^{\ell_\mathsf{r}+\lambda}$, then z can be omitted from ct.

[8] For TDPs, it is typical to consider the case that the domain is dependent on an evaluation key. The following argument with the canonical domain sampling property works even if the underlying TDP has an evaluation-key-dependent domain.

3. Compute $\mathsf{r}'^* \leftarrow \mathsf{hc}(\mathsf{ek}, \mathsf{r}^*)$.
4. Compute $\mathsf{k}'^* \leftarrow \mathsf{G}(\mathsf{r}'^*)$.
5. Parse k'^* as $(\mathsf{k}^*, \mathsf{z}^*) \in \{0,1\}^{4\lambda} \times \{0,1\}^{\lambda}$.
6. Set $\mathsf{ct}^* \leftarrow (\mathsf{y}^*, \mathsf{z}^*)$.

- Run $\mathcal{A}(\mathsf{pk}, \mathsf{ct}^*, \mathsf{k}^*)$.
- At some point, $\mathcal{A}$ outputs $b' \in \{0,1\}$ and terminates.

Game 2: Same as Game 1, except that r'^* is picked uniformly at random from $\{0,1\}^{\lambda}$, instead of being computed as $\mathsf{r}'^* \leftarrow \mathsf{hc}(\mathsf{ek}, \mathsf{r}^*)$.

We can construct a reduction algorithm $\mathcal{B}$ that attacks the security of the hardcore function hc for TDF such that $\mathsf{Adv}^{\mathsf{hc}}_{\mathsf{TDF},\mathsf{hc},\mathcal{B}}(\lambda) = |\Pr[\mathtt{T}_1] - \Pr[\mathtt{T}_2]|$. (The reduction is straightforward and thus omitted.)

Game 3: Same as Game 2, except that now $\mathsf{k}'^* = (\mathsf{k}^*, \mathsf{z}^*)$ is chosen uniformly at random from $\{0,1\}^{4\lambda} \times \{0,1\}^{\lambda}$, instead of being generated as $\mathsf{k}'^* = (\mathsf{k}^*, \mathsf{z}^*) \leftarrow \mathsf{G}(\mathsf{r}'^*)$.

We can construct a reduction algorithm $\mathcal{B}'$ that attacks the security of the PRG G such that $\mathsf{Adv}^{\mathsf{prg}}_{\mathsf{G},\mathcal{B}'}(\lambda) = |\Pr[\mathtt{T}_2] - \Pr[\mathtt{T}_3]|$. (The reduction is again straightforward and omitted.)

Game 4: Same as Game 3, except that now y^* is chosen uniformly at random from $\mathcal{Y}$, instead of being generated as $\mathsf{y}^* \leftarrow \mathsf{Eval}(\mathsf{ek}, \mathsf{r}^*)$.

We can construct a reduction algorithm $\mathcal{B}''$ that attacks the pseudorandomness of the TDF TDF such that $\mathsf{Adv}^{\mathsf{pr}}_{\mathsf{TDF},\mathcal{B}''}(\lambda) = |\Pr[\mathtt{T}_3] - \Pr[\mathtt{T}_4]|$. (The reduction is again straightforward and omitted.)

Note that Game 4 is exactly the pseudorandom ciphertext experiment $\mathsf{Expt}^{\mathsf{prct}}_{\mathsf{KEM}_1,\mathcal{A}}(\lambda)$ in which $b = 0$. Specifically, in Game 4, $\mathsf{ct}^* = (\mathsf{y}^*, \mathsf{z}^*)$ and k^* are chosen uniformly at random, independently of any other values. Hence, this game is exactly the pseudorandom ciphertext experiment with $b = 0$.

By the above arguments and the triangle inequality, we have

$$\begin{aligned}
\mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM}_1,\mathcal{A}}(\lambda) &= 2 \cdot \left|\Pr[\mathsf{Expt}^{\mathsf{prct}}_{\mathsf{KEM}_1,\mathcal{A}}(\lambda) = 1] - \frac{1}{2}\right| \\
&= \left|\Pr[\mathsf{Expt}^{\mathsf{prct}}_{\mathsf{KEM}_1,\mathcal{A}}(\lambda) : b' = 1 | b = 1] - \Pr[\mathsf{Expt}^{\mathsf{prct}}_{\mathsf{KEM}_1,\mathcal{A}}(\lambda) : b' = 1 | b = 0]\right| \\
&= \left|\Pr[\mathtt{T}_1] - \Pr[\mathtt{T}_4]\right| \\
&\leq \left|\Pr[\mathtt{T}_1] - \Pr[\mathtt{T}_2\right| + \left|\Pr[\mathtt{T}_2] - \Pr[\mathtt{T}_3]\right| + \left|\Pr[\mathtt{T}_3] - \Pr[\mathtt{T}_4]\right| \\
&= \mathsf{Adv}^{\mathsf{hc}}_{\mathsf{TDF},\mathsf{hc},\mathcal{B}}(\lambda) + \mathsf{Adv}^{\mathsf{pcg}}_{\mathsf{G},\mathcal{B}'}(\lambda) + \mathsf{Adv}^{\mathsf{pr}}_{\mathsf{TDF},\mathcal{B}''}(\lambda),
\end{aligned}$$

as desired. □ (**Theorem 3**)

3.2 From a One-way TDF and a KEM with the Pseudorandom Ciphertext Property

Here, we show how to construct a randomness-recoverable KEM that satisfies the pseudorandom ciphertext property from the combination of a one-way TDF and a (non-randomness-recoverable) KEM with the pseudorandom ciphertext property.

Let $\mathsf{TDF} = (\mathsf{Setup}, \mathsf{Samp}, \mathsf{Eval}, \mathsf{Inv})$ be a TDF such that its domain (for security parameter λ) is $\{0,1\}^{\ell_{\mathsf{r}}}$ for some polynomial $\ell_{\mathsf{r}} = \ell_{\mathsf{r}}(\lambda)$, Samp is a uniform distribution over $\{0,1\}^{\ell_{\mathsf{r}}}$, the

output length of Eval is some polynomial $\ell_{\mathsf{y}} = \ell_{\mathsf{y}}(\lambda)$, and it has a hardcore function hc whose output length is λ. Let $\mathsf{KEM}' = (\mathsf{KKG}', \mathsf{Encap}', \mathsf{Decap}')$ be a (non-randomness-recoverable) KEM such that the randomness space of Encap' is $\{0,1\}^\lambda$, its session-key space is $\{0,1\}^{4\lambda+\ell_{\mathsf{y}}}$,[9] and the ciphertext length is ℓ_{ct} for some polynomial $\ell_{\mathsf{ct}} = \ell_{\mathsf{ct}}(\lambda)$.

Using TDF and KEM', we construct a randomness-recoverable KEM $\mathsf{KEM}_2 = (\mathsf{KKG}_2, \mathsf{Encap}_2, \mathsf{RDecap}_2)$ as in Figure 3 (left). The ciphertext space is $\{0,1\}^{\ell_{\mathsf{ct}}+\ell_{\mathsf{y}}}$, which is trivially an abelian group whose group operation is bit-wise XOR.

The security of KEM_2 is guaranteed by the following theorem.

Theorem 4 *If* KEM' *satisfies the pseudorandom ciphertext property and* hc *is a hardcore function of* TDF*, then* KEM_2 *satisfies the pseudorandom ciphertext property.*

Proof of Theorem 4. Let $\mathcal{A}$ be any PPT adversary that attacks the pseudorandom ciphertext property of KEM_2. We show that for this $\mathcal{A}$, there exist PPT adversaries $\mathcal{B}$ and $\mathcal{B}'$ that satisfy

$$\mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM}_2,\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\mathsf{hc}}_{\mathsf{TDF},\mathsf{hc},\mathcal{B}}(\lambda) + \mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM}',\mathcal{B}'}(\lambda),$$

which implies the theorem.

Our proof is via a sequence of games argument using three games. In the following, let $\mathtt{T}_t$ denote the event that $\mathcal{A}$ outputs $b' = 1$ in Game $t \in [3]$.

Game 1: This is the pseudorandom ciphertext experiment $\mathsf{Expt}^{\mathsf{prct}}_{\mathsf{KEM}_2,\mathcal{A}}(\lambda)$ in which $b = 1$.

Specifically, the description of Game 1 is as follows:

- Compute $(\mathsf{ek}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda)$ and $(\mathsf{pk}', \mathsf{sk}') \leftarrow \mathsf{KKG}'(1^\lambda)$, and set $\mathsf{pk} \leftarrow (\mathsf{ek}, \mathsf{pk}')$.
- Generate $\mathsf{ct}^* = (\mathsf{ct}^*_1, \mathsf{ct}^*_2)$ and k^* as follows.
  1. Pick $\mathsf{r}^* \xleftarrow{\mathsf{r}} \{0,1\}^{\ell_{\mathsf{r}}}$ uniformly at random.
  2. Compute $\mathsf{y}^* \leftarrow \mathsf{Eval}(\mathsf{ek}, \mathsf{r}^*)$.
  3. Compute $\mathsf{r}'^* \leftarrow \mathsf{hc}(\mathsf{ek}, \mathsf{r}^*)$.
  4. Compute $(\mathsf{ct}^*_1, \mathsf{k}'^*) \leftarrow \mathsf{Encap}'(\mathsf{pk}'; \mathsf{r}'^*)$.
  5. Parse k'^* as $(\mathsf{k}^*, \mathsf{s}^*) \in \{0,1\}^{4\lambda} \times \{0,1\}^{\ell_{\mathsf{y}}}$.
  6. Compute $\mathsf{ct}^*_2 \leftarrow \mathsf{y}^* \oplus \mathsf{s}^*$.
  7. Set $\mathsf{ct}^* \leftarrow (\mathsf{ct}^*_1, \mathsf{ct}^*_2)$.
- Run $\mathcal{A}(\mathsf{pk}, \mathsf{ct}^*, \mathsf{k}^*)$.
- At some point, $\mathcal{A}$ outputs $b' \in \{0,1\}$ and terminates.

Game 2: Same as Game 1, except that r'^* is picked uniformly at random from $\{0,1\}^\lambda$, instead of being computed as $\mathsf{r}'^* \leftarrow \mathsf{hc}(\mathsf{ek}, \mathsf{r}^*)$.

We can construct a reduction algorithm $\mathcal{B}$ that attacks the security of the hardcore function hc for TDF such that $\mathsf{Adv}^{\mathsf{hc}}_{\mathsf{TDF},\mathsf{hc},\mathcal{B}}(\lambda) = |\Pr[\mathtt{T}_1] - \Pr[\mathtt{T}_2]|$. (The reduction is straightforward and thus omitted.)

Game 3: Same as Game 2, except that now $(\mathsf{ct}^*_1, \mathsf{k}'^*)$ is chosen uniformly at random from $\{0,1\}^{\ell_{\mathsf{ct}}} \times \{0,1\}^{4\lambda+\ell_{\mathsf{y}}}$, instead of being generated as $(\mathsf{ct}^*_1, \mathsf{k}'^*) \leftarrow \mathsf{Encap}'(\mathsf{pk}'; \mathsf{r}'^*)$.

We can construct a reduction algorithm $\mathcal{B}'$ that attacks the pseudorandom ciphertext property of KEM' such that $\mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM}',\mathcal{B}'}(\lambda) = |\Pr[\mathtt{T}_2] - \Pr[\mathtt{T}_3]|$. (The reduction is again straightforward and omitted.)

[9] Any KEM with the pseudorandom ciphertext property can be generically turned into a KEM with these properties by appropriately using a PRG.

Note that Game 3 is exactly the pseudorandom ciphertext experiment $\mathsf{Expt}^{\mathsf{prct}}_{\mathsf{KEM}_2,\mathcal{A}}(\lambda)$ in which $b = 0$. Specifically, in Game 3, ct_1^* and $\mathsf{k}'^* = (\mathsf{k}^*, \mathsf{s}^*)$ are chosen randomly, and s^* makes $\mathsf{ct}_2^* = \mathsf{y}^* \oplus \mathsf{s}^*$ distributed uniformly at random in $\{0,1\}^{\ell_\mathsf{y}}$, so that now both $\mathsf{ct}^* = (\mathsf{ct}_1^*, \mathsf{ct}_2^*)$ and k^* are distributed uniformly at random (independently of any other values). Hence, this game is exactly the pseudorandom ciphertext experiment with $b = 0$.

By the above arguments and the triangle inequality, we have

$$\begin{aligned}
\mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM}_2,\mathcal{A}}(\lambda) &= 2 \cdot \left|\Pr[\mathsf{Expt}^{\mathsf{prct}}_{\mathsf{KEM}_2,\mathcal{A}}(\lambda) = 1] - \frac{1}{2}\right| \\
&= \left|\Pr[\mathsf{Expt}^{\mathsf{prct}}_{\mathsf{KEM}_2,\mathcal{A}}(\lambda) : b' = 1 | b = 1] - \Pr[\mathsf{Expt}^{\mathsf{prct}}_{\mathsf{KEM}_2,\mathcal{A}}(\lambda) : b' = 1 | b = 0]\right| \\
&= \left|\Pr[\mathtt{T}_1] - \Pr[\mathtt{T}_3]\right| \\
&\leq \left|\Pr[\mathtt{T}_1] - \Pr[\mathtt{T}_2]\right| + \left|\Pr[\mathtt{T}_2] - \Pr[\mathtt{T}_3]\right| \\
&= \mathsf{Adv}^{\mathsf{hc}}_{\mathsf{TDF},\mathsf{hc},\mathcal{B}}(\lambda) + \mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM}',\mathcal{B}'}(\lambda),
\end{aligned}$$

as desired. □ (**Theorem 4**)

4 Our Proposed TDF Construction

In this section, we show our proposed TDF with adaptive one-wayness and adaptive pseudorandomness. Since we have explained the technical overview of our proposed TDF construction in Section 1.4, we directly proceed to the construction. Specifically, in Section 4.1, we present the formal description of our proposed TDF, state theorems regarding correctness and security, and give some remarks on the properties of our construction. Then, in Sections 4.2 and 4.3, we prove the correctness and security of our proposed construction, respectively.

4.1 Our Construction

Our TDF uses the following building blocks:

- Let $\mathsf{KEM} = (\mathsf{KKG}, \mathsf{Encap}, \mathsf{Decap}, \mathsf{RDecap})$ be a randomness-recoverable KEM such that (1) its session-key space is $\{0,1\}^{4\lambda}$; (2) the randomness space of Encap is $\{0,1\}^{\ell_\mathsf{r}}$ for some polynomial $\ell_\mathsf{r} = \ell_\mathsf{r}(\lambda)$; (3) the ciphertext space $\mathcal{C}$ forms an abelian group (where we use the additive notation) and satisfies $|\mathcal{C}| \geq 2^{\ell_\mathsf{r}+\lambda}$; (4) the number of possible session-keys k that could be output by $\mathsf{Encap}(\mathsf{pk};\cdot)$ for any pk output by $\mathsf{KKG}(1^\lambda)$ is at most 2^λ.
- Let $\mathsf{Hash} = (\mathsf{HKG}, \mathsf{H})$ be a keyed hash function such that the range of H is $\{0,1\}^\lambda$, which we are going to assume to be target collision resistant.

Furthermore, let $n = \ell_\mathsf{r} + 2\lambda$.

Using these building blocks, the proposed TDF $\mathsf{ATDF} = (\mathsf{Setup}, \mathsf{Samp}, \mathsf{Eval}, \mathsf{Inv})$ is constructed as described in Figure 4. The domain $\mathcal{X}$ and range $\mathcal{Y}$ of ATDF are given by $\mathcal{X} = \{0,1\}^n \times \{0,1\}^{n \cdot \ell_\mathsf{r}}$ and $\mathcal{Y} = \mathcal{C} \times (\mathcal{C} \times \{0,1\}^{4\lambda})^n$, respectively. Since we are assuming $|\mathcal{C}| \geq 2^{\ell_\mathsf{r}+\lambda}$, $|\mathcal{X}|$ is exponentially smaller than $|\mathcal{Y}|$.

We note that in our TDF construction, k_0 (corresponding to ct_0) can be used as the hardcore bits of the input x. We can also interpret our TDF as a randomness-recoverable KEM in which y is a ciphertext and k_0 is the corresponding session-key. Interpreted as a randomness-recoverable KEM, the adaptive pseudorandomness of the TDF directly translates to the pseudorandom

$\mathsf{Setup}(1^\lambda)$:
$\forall i \in \{0\} \cup [n]$:
$(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KKG}(1^\lambda)$
$\mathsf{A}_1, \ldots, \mathsf{A}_n, \mathsf{B} \xleftarrow{\mathsf{r}} \{0,1\}^{4\lambda}$
$\mathsf{C}_1, \ldots, \mathsf{C}_n \xleftarrow{\mathsf{r}} \mathcal{C}$
$\mathsf{hk} \leftarrow \mathsf{HKG}(1^\lambda)$
$\mathsf{ek} \leftarrow ((\mathsf{pk}_i, \mathsf{A}_i, \mathsf{C}_i)_{i\in[n]}, \mathsf{pk}_0, \mathsf{B}, \mathsf{hk})$
$\mathsf{td} \leftarrow ((\mathsf{sk}_i)_{i\in[n]}, \mathsf{ek})$
Return $(\mathsf{ek}, \mathsf{td})$.

$\mathsf{Samp}(1^\lambda)$:
$\mathsf{s} = (\mathsf{s}_1, \ldots, \mathsf{s}_n) \xleftarrow{\mathsf{r}} \{0,1\}^n$
$\mathsf{r}_1^{\mathsf{s}_1}, \ldots, \mathsf{r}_n^{\mathsf{s}_n} \xleftarrow{\mathsf{r}} \{0,1\}^{\ell_\mathsf{r}}$
Return $\mathsf{x} \leftarrow (\mathsf{s}, (\mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]})$.

$\mathsf{Eval}(\mathsf{ek}, \mathsf{x})$:
$((\mathsf{pk}_i, \mathsf{A}_i, \mathsf{C}_i)_{i\in[n]}, \mathsf{pk}_0, \mathsf{B}, \mathsf{hk}) \leftarrow \mathsf{ek}$
$(\mathsf{s} = (\mathsf{s}_1, \ldots, \mathsf{s}_n), (\mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]}) \leftarrow \mathsf{x}$
$\forall i \in [n]$:
$(\mathsf{ct}_i^{\mathsf{s}_i}, \mathsf{k}_i^{\mathsf{s}_i}) \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{\mathsf{s}_i})$
$\mathsf{ct}_i \leftarrow \mathsf{ct}_i^{\mathsf{s}_i} + \mathsf{s}_i \cdot \mathsf{C}_i$ (†)
$= \begin{cases} \mathsf{ct}_i^0 & \text{if } \mathsf{s}_i = 0 \\ \mathsf{ct}_i^1 + \mathsf{C}_i & \text{if } \mathsf{s}_i = 1 \end{cases}$
$\mathsf{r}_0 \leftarrow \bigoplus_{i\in[n]} \mathsf{r}_i^{\mathsf{s}_i}$
$(\mathsf{ct}_0, \mathsf{k}_0) \leftarrow \mathsf{Encap}(\mathsf{pk}_0; \mathsf{r}_0)$
$\mathsf{h} \leftarrow \mathsf{H}(\mathsf{hk}, \mathsf{ct}_0 \| (\mathsf{ct}_i)_{i\in[n]})$
$\forall i \in [n]$:
$\mathsf{T}_i \leftarrow \mathsf{k}_i^{\mathsf{s}_i} + \mathsf{s}_i \cdot (\mathsf{A}_i + \mathsf{B} \cdot \mathsf{h})$
$= \begin{cases} \mathsf{k}_i^0 & \text{if } \mathsf{s}_i = 0 \\ \mathsf{k}_i^1 + \mathsf{A}_i + \mathsf{B} \cdot \mathsf{h} & \text{if } \mathsf{s}_i = 1 \end{cases}$
Return $\mathsf{y} \leftarrow (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$.

$\mathsf{Inv}(\mathsf{td}, \mathsf{y})$:
$((\mathsf{sk}_i)_{i\in[n]}, \mathsf{ek}) \leftarrow \mathsf{td}$
$((\mathsf{pk}_i, \mathsf{A}_i, \mathsf{C}_i)_{i\in[n]}, \mathsf{pk}_0, \mathsf{B}, \mathsf{hk}) \leftarrow \mathsf{ek}$
$(\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]}) \leftarrow \mathsf{y}$
$\mathsf{h} \leftarrow \mathsf{H}(\mathsf{hk}, \mathsf{ct}_0 \| (\mathsf{ct}_i)_{i\in[n]})$
$\forall (i, v) \in [n] \times \{0,1\}$:
$(\mathsf{k}_i^v, \mathsf{r}_i^v) \leftarrow \mathsf{RDecap}(\mathsf{sk}_i, \mathsf{ct}_i - v \cdot \mathsf{C}_i)$ (†)
$\mathsf{chk}_i^v \leftarrow \begin{cases} 1 & \text{if } (\mathsf{k}_i^v, \mathsf{r}_i^v) \neq \bot \wedge \\ & \quad \mathsf{T}_i = \mathsf{k}_i^v + v \cdot (\mathsf{A}_i + \mathsf{B} \cdot \mathsf{h}) \\ 0 & \text{otherwise} \end{cases}$
If $\exists i \in [n] : \mathsf{chk}_i^0 = \mathsf{chk}_i^1$ then return $\bot$.
$\forall i \in [n] : \mathsf{s}_i \leftarrow \mathsf{chk}_i^1$
$\mathsf{r}_0 \leftarrow \bigoplus_{i\in[n]} \mathsf{r}_i^{\mathsf{s}_i}$
If $\mathsf{Encap}(\mathsf{pk}_0; \mathsf{r}_0) \neq (\mathsf{ct}_0, *)$
then return $\bot$.
Return $\mathsf{x} \leftarrow (\mathsf{s} = (\mathsf{s}_1, \ldots, \mathsf{s}_n), (\mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]})$.

Figure 4: The proposed TDF ATDF. Arithmetic involving $\mathsf{h} \in \{0,1\}^\lambda$ treats it as an element of $\{0,1\}^{4\lambda}$ by some canonical injective encoding (say, putting the prefix $0^{3\lambda}$), and the arithmetic is done over $\mathrm{GF}(2^{4\lambda})$ where we identify $\{0,1\}^{4\lambda}$ with $\mathrm{GF}(2^{4\lambda})$. (†) The addition/subtraction is done over $\mathcal{C}$.

ciphertext property of the KEM under chosen-ciphertext attacks (i.e. the pseudorandom ciphertext property holds even in the presence of the decapsulation oracle).

For the correctness and security of ATDF, the following theorems hold.

Theorem 5 *Let $\epsilon = \epsilon(\lambda) \in [0,1]$. If* KEM *is ϵ-almost-all-keys correct, then* ATDF *is $n\cdot(\epsilon+2^{-\lambda})$-almost-all-keys correct.*

Theorem 6 *Assume that* KEM *satisfies the pseudorandom ciphertext property and almost-all-keys correctness, and* Hash *is target collision resistant. Then,* ATDF *satisfies adaptive one-wayness and adaptive pseudorandomness.*

The proofs of Theorems 5 and 6 are given in Sections 4.2 and 4.3, respectively.

As we showed in Section 3.2, the building block KEM used in our construction can be built from the combination of a one-way TDF and a PKE scheme/KEM with the pseudorandom ciphertext property. Furthermore, a target collision resistant hash function can be built from any one-way function, whose existence is in turn implied by a one-way TDF. Hence, combined

with Theorems 5 and 6, we obtain the formal version of Theorem 1, as our main result in this paper.

Theorem 7 *Assume that there exist a one-way TDF and a PKE scheme/KEM with the pseudorandom ciphertext property. Then, there exists a TDF that satisfies adaptive one-wayness and adaptive pseudorandomness.*

Furthermore, by instantiating the building block KEM from a TDP with canonical domain sampling as explained in Section 3.1, and combining Theorems 5 and 6, we obtain the formal version of Corollary 1.

Corollary 2 *Assume that there exist a one-way TDP with the canonical domain sampling property [GLN11]. Then, there exists a TDF that satisfies adaptive one-wayness and adaptive pseudorandomness.*

Tag-Based Adaptive Trapdoor Function Induced from Our TDF Construction. As explained in Section 1.4, our TDF construction incorporates a tag-based adaptive TDF in its structure, where instead of computing h as a hash of the components $\mathsf{ct}_0\|(\mathsf{ct}_i)_{i\in[n]}$, we set an arbitrary string for h as a tag in a tag-based TDF. Then, this tag-based TDF straightforwardly induced from our TDF satisfies adaptive one-wayness (of a tag-based TDF) as defined in [KMO10], which will be evident from our security proof in Section 4.3. In this paper, we would like to achieve adaptive pseudorandomness for our TDF, which has several applications as explained in the introduction. However, it is unknown if tag-based TDFs are sufficient for the applications.[10] Hence, we do not explore this direction further.

On Adaptive Trapdoor Permutations. Given our result, one natural question is whether our result can be extended to achieve adaptive trapdoor *permutations*. Unfortunately, our adaptive TDF construction does not seem to be useful for achieving an adaptive TDP, since an image value has to include redundant components that are used for the validity checking in the inversion algorithm. We believe that it is an interesting open question to seek for a generic construction of an adaptive TDP from a weaker primitive (say, one-way TDP).

4.2 Proof of Correctness (Proof of Theorem 5)

Let $\mathsf{ek} = ((\mathsf{pk}_i, \mathsf{A}_i, \mathsf{C}_i)_{i\in[n]}, \mathsf{pk}_0, \mathsf{B}, \mathsf{hk})$ be an evaluation key and $\mathsf{td} = ((\mathsf{sk}_i)_{i\in[n]}, \mathsf{ek})$ be the corresponding trapdoor output by $\mathsf{Setup}(1^\lambda)$. Using the values in $(\mathsf{ek}, \mathsf{td})$, for each $i \in [n]$, we define the function $f_i : \{0,1\}^{\ell_r} \times \{0,1\}^\lambda \to \{0,1\}^{4\lambda} \cup \{\bot\}$ by

$$f_i(\mathsf{r}, \mathsf{h}) : \left[\begin{array}{l} (\mathsf{ct}, \mathsf{k}) \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r});\ (\mathsf{k}', \mathsf{r}') \leftarrow \mathsf{RDecap}(\mathsf{sk}_i, \mathsf{ct} - \mathsf{C}_i); \\ \text{If } (\mathsf{k}', \mathsf{r}') = \bot \text{ then return } \bot \text{ else return } \mathsf{k} - \mathsf{k}' - \mathsf{B} \cdot \mathsf{h} \end{array} \right].$$

We say that an evaluation key/trapdoor pair $(\mathsf{ek}, \mathsf{td})$ is *bad* if (1) some of $\{\mathsf{pk}_i\}_{i\in[n]}$ is erroneous, or (2) there exists $i \in [n]$ such that A_i belongs to the image of f_i. Otherwise, we say that $(\mathsf{ek}, \mathsf{td})$ is *good*. Since the number of possible session-keys output by Encap (for any public key output by $\mathsf{KKG}(1^\lambda)$) is assumed to be at most 2^λ, k and k' can each take at most 2^λ values. Hence, the image size of each f_i (excluding $\bot$) is at most $2^{3\lambda}$. Since each A_i is chosen uniformly at

[10] Adaptive one-wayness for a tag-based TDF only guarantees that one-wayness under a challenge tag h^* cannot be broken if an adversary has access to the inversion oracle for tags that are different from the challenge tag h^*. (See [KMO10] for its formal definition.) Thus, it does not rule out the possibility that breaking one-wayness might be easy if the adversary has access to the inversion oracle under the challenge tag h^*, which is why it does not immediately lead to an ordinary (non-tag-based) adaptive TDF.

random from $\{0,1\}^{4\lambda}$, the probability that $\mathsf{Setup}(1^\lambda)$ outputs a bad pair $(\mathsf{ek},\mathsf{td})$ is at most $n\cdot\epsilon + n\cdot\frac{2^{3\lambda}}{2^{4\lambda}} = n\cdot(\epsilon+2^{-\lambda})$ by the union bound.

Now, consider the case that a good pair $(\mathsf{ek},\mathsf{td})$ is output by Setup. Let $\mathsf{x} = (\mathsf{s} = (\mathsf{s}_1,\dots,\mathsf{s}_n),(\mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]}) \in \{0,1\}^{n+n\ell_\mathsf{r}}$ be a domain element of ATDF, and let $\mathsf{y} = (\mathsf{ct}_0,(\mathsf{ct}_i,\mathsf{T}_i)_{i\in[n]}) = \mathsf{Eval}(\mathsf{ek},\mathsf{x})$. Let $\mathsf{h} = \mathsf{H}(\mathsf{hk},\mathsf{ct}_0\|(\mathsf{ct}_i)_{i\in[n]})$. Then, consider an execution of $\mathsf{Inv}(\mathsf{td},\mathsf{y})$. Let $(\mathsf{k}_i'^v,\mathsf{r}_i'^v) = \mathsf{RDecap}(\mathsf{sk}_i,\mathsf{ct}_i - v\cdot\mathsf{C}_i)$ for all $(i,v)\in[n]\times\{0,1\}$.[11] Below, we denote the values computed in Inv with an apostrophe. Similarly, we denote the values computed in Eval without an apostrophe.

By the design of Eval, $\mathsf{ct}_i = \mathsf{ct}_i^{\mathsf{s}_i} + \mathsf{s}_i\cdot\mathsf{C}_i$ holds for all $i\in[n]$. Hence, for all $i\in[n]$, we have $(\mathsf{k}_i'^{\mathsf{s}_i},\mathsf{r}_i'^{\mathsf{s}_i}) = \mathsf{RDecap}(\mathsf{sk}_i,\mathsf{ct}_i - \mathsf{s}_i\cdot\mathsf{C}_i) = \mathsf{RDecap}(\mathsf{sk}_i,\mathsf{ct}_i^{\mathsf{s}_i}) = (\mathsf{k}_i^{\mathsf{s}_i},\mathsf{r}_i^{\mathsf{s}_i})$. This in turn implies $\mathsf{T}_i = \mathsf{k}_i'^{\mathsf{s}_i} + \mathsf{s}_i\cdot(\mathsf{A}_i + \mathsf{B}\cdot\mathsf{h})$ due to how it is computed in Eval. Thus, Inv sets $\mathsf{chk}_i'^{\mathsf{s}_i}\leftarrow 1$.

We argue that for all $i\in[n]$, either $(\mathsf{k}_i'^{1-\mathsf{s}_i},\mathsf{r}_i'^{1-\mathsf{s}_i}) = \bot$ or $\mathsf{T}_i \neq \mathsf{k}'^{1-\mathsf{s}_i} + (1-\mathsf{s}_i)\cdot(\mathsf{A}_i+\mathsf{B}\cdot\mathsf{h})$ holds, and thus Inv sets $\mathsf{chk}_i'^{1-\mathsf{s}_i}\leftarrow 0$. To this end, assume towards a contradiction that there exists $j\in[n]$ for which $(\mathsf{k}_j'^{1-\mathsf{s}_j},\mathsf{r}_j'^{1-\mathsf{s}_j})\neq\bot$ and $\mathsf{T}_j = \mathsf{k}'^{1-\mathsf{s}_j} + (1-\mathsf{s}_j)\cdot(\mathsf{A}_j+\mathsf{B}\cdot\mathsf{h})$ hold simultaneously. Then, the equalities regarding T_j yield

$$\mathsf{k}_j'^{\mathsf{s}_j} + \mathsf{s}_j\cdot(\mathsf{A}_j+\mathsf{B}\cdot\mathsf{h}) = \mathsf{k}_j'^{1-\mathsf{s}_j} + (1-\mathsf{s}_j)\cdot(\mathsf{A}_j+\mathsf{B}\cdot\mathsf{h}) \iff \mathsf{k}_j'^0 = \mathsf{k}_j'^1 + \mathsf{A}_j + \mathsf{B}\cdot\mathsf{h}. \tag{3}$$

On the other hand, since A_j is not in the image of f_j, we have

$$\mathsf{A}_j \neq f_j(\mathsf{r}_j'^0,\mathsf{h}) \overset{(*)}{=} \mathsf{k}_j'^0 - \mathsf{k}_j'^1 - \mathsf{B}\cdot\mathsf{h} \iff \mathsf{k}_j'^0 \neq \mathsf{k}_j'^1 + \mathsf{A}_j + \mathsf{B}\cdot\mathsf{h}, \tag{4}$$

where the equality (*) uses the fact that $\mathsf{Encap}(\mathsf{pk}_j;\mathsf{r}_j'^v) = (\mathsf{ct}_j - v\cdot\mathsf{C}_j,\mathsf{k}_j'^v)$ holds for both $v\in\{0,1\}$ due to the correctness of KEM under a non-erroneous key. However, Eqs. (3) and (4) contradict each other.

Hence, for each $i\in[n]$, we have $(\mathsf{chk}_i'^{\mathsf{s}_i},\mathsf{chk}_i'^{1-\mathsf{s}_i}) = (1,0) \Leftrightarrow (\mathsf{chk}_i'^0,\mathsf{chk}_i'^1) = (1-\mathsf{s}_i,\mathsf{s}_i)$, which in turn implies that we have $\mathsf{s}_i' = \mathsf{chk}_i'^1 = \mathsf{s}_i$. Since $\mathsf{r}_i'^{\mathsf{s}_i} = \mathsf{r}_i^{\mathsf{s}_i}$ holds for all $i\in[n]$, we have $\mathsf{r}_0' = \bigoplus_{i\in[n]}\mathsf{r}_i'^{\mathsf{s}_i'} = \bigoplus_{i\in[n]}\mathsf{r}_i^{\mathsf{s}_i} = \mathsf{r}_0$. Hence, the validity check of ct_0 performed at the second last step of Inv never fails, and Inv will output $\mathsf{x}' = (\mathsf{s}' = (\mathsf{s}_1',\dots,\mathsf{s}_n'),(\mathsf{r}_i'^{\mathsf{s}_i'})_{i\in[n]}) = (\mathsf{s} = (\mathsf{s}_1,\dots,\mathsf{s}_n),(\mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]}) = \mathsf{x}$.

Putting everything together, except for probability at most $n\cdot(\epsilon+2^{-\lambda})$ over $(\mathsf{ek},\mathsf{td})\leftarrow\mathsf{Setup}(1^\lambda)$, $\mathsf{Inv}(\mathsf{td},\mathsf{Eval}(\mathsf{ek},\mathsf{x})) = \mathsf{x}$ holds for all $\mathsf{x}\in\{0,1\}^{n+n\ell_\mathsf{r}}$. □ (**Theorem 5**)

4.3 Proof of Security (Proof of Theorem 6)

As mentioned earlier, $|\mathcal{X}|$ is exponentially smaller than $|\mathcal{Y}|$ and thus, by Theorem 2, it is sufficient to show that ATDF satisfies adaptive pseudorandomness.

Let $\epsilon = \epsilon(\lambda)$ be such that KEM is ϵ-almost-all-keys correct. Let $\mathcal{A}$ be any PPT adversary that attacks the adaptive pseudorandomness of ATDF. We will show that for this $\mathcal{A}$, there exist PPT adversaries $\mathcal{B}_\mathsf{h}$, $\mathcal{B}_\mathsf{h}'$, $\mathcal{B}$, $\mathcal{B}'$, and $\mathcal{B}''$ that satisfy

$$\begin{aligned}\mathsf{Adv}^{\mathsf{apr}}_{\mathsf{ATDF},\mathcal{A}}(\lambda) \le{}& \mathsf{Adv}^{\mathsf{tcr}}_{\mathsf{Hash},\mathcal{B}_\mathsf{h}}(\lambda) + \mathsf{Adv}^{\mathsf{tcr}}_{\mathsf{Hash},\mathcal{B}_\mathsf{h}'}(\lambda) + n\cdot\mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM},\mathcal{B}}(\lambda) + \mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM},\mathcal{B}'}(\lambda)\\ &+ 2n\cdot\mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM},\mathcal{B}''}(\lambda) + (4n^2+5n)\epsilon + (4n^2+n+2)\cdot 2^{-\lambda},\end{aligned} \tag{5}$$

which implies the theorem.

Our proof is via a sequence of games argument using the following eight games.

[11] Note that $(\mathsf{k}_i'^v,\mathsf{r}_i'^v)$ could be $\bot$ in case the decapsulated ciphertext $\mathsf{ct}_i - v\cdot\mathsf{C}_i$ is invalid.

Game 1: This is the adaptive pseudorandomness experiment $\mathsf{Expt}^{\mathsf{apr}}_{\mathsf{ATDF},\mathcal{A}}(\lambda)$ in which $b = 1$. However, for making it easier to describe the subsequent games, we change the ordering of the operations for how the evaluation key/trapdoor pair $(\mathsf{ek}, \mathsf{td})$ and the challenge instance $\mathsf{y}^* = (\mathsf{ct}_0^*, (\mathsf{ct}_i^*, \mathsf{T}_i^*)_{i\in[n]}) = \mathsf{Eval}(\mathsf{ek}, \mathsf{x}^*)$ are generated, so that the distribution of $(\mathsf{ek}, \mathsf{td}, \mathsf{y}^*)$ is identical to that in the original adaptive pseudorandomness experiment with $b = 1$.

Specifically, the description of Game 1 is as follows:

- Generate $\mathsf{ek} = ((\mathsf{pk}_i, \mathsf{A}_i, \mathsf{C}_i)_{i\in[n]}, \mathsf{pk}_0, \mathsf{B}, \mathsf{hk})$, $\mathsf{td} = ((\mathsf{sk}_i)_{i\in[n]}, \mathsf{ek})$, and $\mathsf{y}^* = (\mathsf{ct}_0^*, (\mathsf{ct}_i^*, \mathsf{T}_i^*)_{i\in[n]})$ as follows:
  1. Compute $(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KKG}(1^\lambda)$ for all $i \in \{0\} \cup [n]$, and pick $\mathsf{B} \xleftarrow{\mathsf{r}} \{0,1\}^{4\lambda}$.
  2. Pick $\mathsf{s}_i^* = (\mathsf{s}_1^*, \ldots, \mathsf{s}_n^*) \xleftarrow{\mathsf{r}} \{0,1\}^n$ and $\mathsf{r}_1^{*0}, \ldots, \mathsf{r}_n^{*0}, \mathsf{r}_1^{*1}, \ldots, \mathsf{r}_n^{*1} \xleftarrow{\mathsf{r}} \{0,1\}^{\ell_\mathsf{r}}$,[12] and set $\mathsf{x}^* \leftarrow (\mathsf{s}^*, (\mathsf{r}_i^{*(\mathsf{s}_i^*)})_{i\in[n]})$.
  3. Compute $(\mathsf{ct}_i^{*(\mathsf{s}_i^*)}, \mathsf{k}_i^{*(\mathsf{s}_i^*)}) \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{*(\mathsf{s}_i^*)})$ for every $i \in [n]$.
  4. Pick $\mathsf{C}_1, \ldots, \mathsf{C}_n \xleftarrow{\mathsf{r}} \mathcal{C}$.
  5. Compute $\mathsf{ct}_i^* \leftarrow \mathsf{ct}_i^{*(\mathsf{s}_i^*)} + \mathsf{s}_i^* \cdot \mathsf{C}_i$ for every $i \in [n]$.
  6. Pick $\mathsf{A}_1, \ldots, \mathsf{A}_n \xleftarrow{\mathsf{r}} \{0,1\}^{4\lambda}$.
  7. Set $\mathsf{r}_0^* \leftarrow \bigoplus_{i\in[n]} \mathsf{r}_i^{*(\mathsf{s}_i^*)}$ and compute $(\mathsf{ct}_0^*, \mathsf{k}_0^*) \leftarrow \mathsf{Encap}(\mathsf{pk}_0; \mathsf{r}_0^*)$.
  8. Compute $\mathsf{hk} \leftarrow \mathsf{HKG}(1^\lambda)$ and $\mathsf{h}^* \leftarrow \mathsf{H}(\mathsf{hk}, \mathsf{ct}_0^* \| (\mathsf{ct}_i^*)_{i\in[n]})$.
  9. Compute $\mathsf{T}_i^* \leftarrow \mathsf{k}_i^{*(\mathsf{s}_i^*)} + \mathsf{s}_i^* \cdot (\mathsf{A}_i + \mathsf{B} \cdot \mathsf{h}^*)$ for every $i \in [n]$.
  10. Set $\mathsf{ek} \leftarrow ((\mathsf{pk}_i, \mathsf{A}_i, \mathsf{C}_i)_{i\in[n]}, \mathsf{pk}_0, \mathsf{B}, \mathsf{hk})$, $\mathsf{td} \leftarrow ((\mathsf{sk}_i)_{i\in[n]}, \mathsf{ek})$, and $\mathsf{y}^* \leftarrow (\mathsf{ct}_0^*, (\mathsf{ct}_i^*, \mathsf{T}_i^*)_{i\in[n]})$.
- Run $\mathcal{A}(\mathsf{ek}, \mathsf{y}^*)$. From here on, $\mathcal{A}$ may start making inversion queries $\mathsf{y} = (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$, which are answered with $\mathsf{x} \leftarrow \mathsf{Inv}(\mathsf{td}, \mathsf{y})$.
- At some point, $\mathcal{A}$ outputs $b' \in \{0,1\}$ and terminates.

Game 2: Same as Game 1, except for an additional rejection rule in the inversion oracle. Specifically, in this game, if $\mathcal{A}$'s inversion query $\mathsf{y} = (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$ satisfies $\mathsf{h} = \mathsf{H}(\mathsf{hk}, \mathsf{ct}_0 \| (\mathsf{ct}_i)_{i\in[n]}) = \mathsf{h}^*$, then the inversion oracle immediately returns $\perp$ to $\mathcal{A}$.

Game 3: Same as Game 2, except for how $(\mathsf{C}_i)_{i\in[n]}$ and $(\mathsf{A}_i)_{i\in[n]}$ are generated.

Specifically, in this game, we pick $(\mathsf{ct}_i^{*(1-\mathsf{s}_i^*)}, \mathsf{k}_i^{*(1-\mathsf{s}_i^*)}) \xleftarrow{\mathsf{r}} \mathcal{C} \times \{0,1\}^{4\lambda}$ for every $i \in [n]$. Then, C_i's and A_i's are generated by

$$\mathsf{C}_i \leftarrow \mathsf{ct}_i^{*0} - \mathsf{ct}_i^{*1} \qquad \text{and} \qquad \mathsf{A}_i \leftarrow \mathsf{k}_i^{*0} - \mathsf{k}_i^{*1} - \mathsf{B} \cdot \mathsf{h}^*. \tag{6}$$

Note that due to the change made in this game, for every $i \in [n]$, we always have $\mathsf{ct}_i^* = \mathsf{ct}_i^{*0}$ and $\mathsf{T}_i^* = \mathsf{k}_i^{*0}$, no matter whether $\mathsf{s}_i^* = 0$ or $\mathsf{s}_i^* = 1$. Indeed, this is the case if $\mathsf{s}_i^* = 0$. Furthermore, if $\mathsf{s}_i^* = 1$, then we have

$$\begin{aligned}
\mathsf{ct}_i^* &= \mathsf{ct}_i^{*1} + \mathsf{C}_i = \mathsf{ct}_i^{*1} + (\mathsf{ct}_i^{*0} - \mathsf{ct}_i^{*1}) = \mathsf{ct}_i^{*0}, \\
\mathsf{T}_i^* &= \mathsf{k}_i^{*1} + \mathsf{A}_i + \mathsf{B} \cdot \mathsf{h}^* = \mathsf{k}_i^{*1} + (\mathsf{k}_i^{*0} - \mathsf{k}_i^{*1} - \mathsf{B} \cdot \mathsf{h}^*) + \mathsf{B} \cdot \mathsf{h}^* = \mathsf{k}_i^{*0}.
\end{aligned}$$

Game 4: Same as Game 3, except that for every $i \in [n]$, $(\mathsf{ct}_i^{*(1-\mathsf{s}_i^*)}, \mathsf{k}_i^{*(1-\mathsf{s}_i^*)})$ is generated as $(\mathsf{ct}_i^{*(1-\mathsf{s}_i^*)}, \mathsf{k}_i^{*(1-\mathsf{s}_i^*)}) \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{*(1-\mathsf{s}_i^*)})$, instead of being picked uniformly at random from $\mathcal{C} \times \{0,1\}^{4\lambda}$.

[12] The values $\{\mathsf{r}_i^{*1-\mathsf{s}_i^*}\}_{i\in[n]}$ are not used in Game 1. Looking ahead, these will be used from Game 4.

Note that due to the change made in this game, every pair $(\mathsf{ct}_i^{*v}, \mathsf{k}_i^{*v})$ is now computed as $(\mathsf{ct}_i^{*v}, \mathsf{k}_i^{*v}) \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{*v})$. Hence, in this game, the only value that is still dependent on $\mathsf{s}^* = (\mathsf{s}_1^*, \ldots, \mathsf{s}_n^*)$, is $\mathsf{r}_0^* = \bigoplus_{i\in[n]} \mathsf{r}_i^{*(\mathsf{s}_i^*)}$.

Game 5: Same as Game 4, except that r_0^* is picked uniformly at random from $\{0,1\}^{\ell_r}$, instead of being computed as $\mathsf{r}_0^* \leftarrow \bigoplus_{i\in[n]} \mathsf{r}_i^{*(\mathsf{s}_i^*)}$.

Due to the change made in this game, r_0^* and all of $\{\mathsf{r}_i^{*v}\}_{i\in[n],v\in\{0,1\}}$ are chosen uniformly at random, and used only for computing $(\mathsf{ct}_0^*, \mathsf{k}_0^*)$ and $\{(\mathsf{ct}_i^{*v}, \mathsf{k}_i^{*v})\}_{i\in[n],v\in\{0,1\}}$, respectively.

Game 6: Same as Game 5, except that ct_0^* is chosen uniformly at random from $\mathcal{C}$, instead of being generated as $(\mathsf{ct}_0^*, \mathsf{k}_0^*) \leftarrow \mathsf{Encap}(\mathsf{pk}_0; \mathsf{r}_0^*)$.

Game 7: Same as Game 6, except that for every $(i, v) \in [n] \times \{0,1\}$, $(\mathsf{ct}_i^{*v}, \mathsf{k}_i^{*v})$ is chosen uniformly at random from $\mathcal{C} \times \{0,1\}^{4\lambda}$, instead of being generated as $(\mathsf{ct}_i^{*v}, \mathsf{k}_i^{*v}) \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{*v})$.

Due to the change made in this game, all the values $\mathsf{A}_1, \ldots, \mathsf{A}_n \in \{0,1\}^{4\lambda}$ and $\mathsf{C}_1, \ldots, \mathsf{C}_n \in \mathcal{C}$ contained in ek, and all of $\mathsf{ct}_0^*, \ldots, \mathsf{ct}_n^* \in \mathcal{C}$ and $\mathsf{T}_i^* \in \{0,1\}^{4\lambda}$ contained in y^*, are chosen uniformly at random, independently with one another. Specifically, for each $i \in [n]$, ct_i^{*1} and k_i^{*1} make C_i and A_i uniformly at random, and in particular completely hide the information about ct_i^{*0} and k_i^{*0} from C_i and A_i, respectively. Then, $\mathsf{ct}_i^* = \mathsf{ct}_i^{*0}$ and $\mathsf{T}_i^* = \mathsf{k}_i^{*0}$ are distributed uniformly at random due to the random choice of ct_i^{*0} and k_i^{*0}, respectively. Therefore, in this game, $(\mathsf{ek}, \mathsf{td})$ is generated exactly as $(\mathsf{ek}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda)$, and y^* is distributed uniformly over $\mathcal{Y} = \mathcal{C} \times (\mathcal{C} \times \{0,1\}^{4\lambda})^n$.

Game 8: Same as Game 7, except that we cancel the rejection rule regarding h^* introduced in Game 2.

Due to the change made in this game, the inversion oracle behaves in exactly the same way as that in the adaptive pseudorandomness experiment. Hence, this game is exactly the adaptive pseudorandomness experiment $\mathsf{Expt}^{\mathsf{apr}}_{\mathsf{ATDF},\mathcal{A}}(\lambda)$ with $b = 0$.

For $t \in [8]$, let $\mathtt{T}_t$ be the event that $\mathcal{A}$ outputs $b' = 1$ in Game t. Then, by the definition of the events and the triangle inequality, we have

$$\begin{aligned}
\mathsf{Adv}^{\mathsf{apr}}_{\mathsf{ATDF},\mathcal{A}}(\lambda) &= 2 \cdot \left|\Pr[\mathsf{Expt}^{\mathsf{apr}}_{\mathsf{ATDF},\mathcal{A}}(\lambda) = 1] - \frac{1}{2}\right| \\
&= \left|\Pr[\mathsf{Expt}^{\mathsf{apr}}_{\mathsf{ATDF},A}(\lambda) : b' = 1 | b = 1] - \Pr[\mathsf{Expt}^{\mathsf{apr}}_{\mathsf{ATDF},A}(\lambda) : b' = 1 | b = 0]\right| \\
&= \left|\Pr[\mathtt{T}_1] - \Pr[\mathtt{T}_8]\right| \\
&\leq \sum_{t\in[7]} \left|\Pr[\mathtt{T}_t] - \Pr[\mathtt{T}_{t+1}]\right|. \qquad (7)
\end{aligned}$$

In the following, we show how the terms appearing in Eq. (7) are bounded.

Lemma 2 *There exists a PPT adversary $\mathcal{B}_{\mathsf{h}}$ such that*

$$\left|\Pr[\mathtt{T}_1] - \Pr[\mathtt{T}_2]\right| \leq \mathsf{Adv}^{\mathsf{tcr}}_{\mathsf{Hash},\mathcal{B}_{\mathsf{h}}}(\lambda) + n \cdot (\epsilon + 2^{-\lambda}).$$

Proof of Lemma 2. We say that an inversion query $\mathsf{y} = (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$ made by $\mathcal{A}$ in Game $j \in [2]$ is *hash-bad* if $\mathsf{h} = \mathsf{H}(\mathsf{hk}, \mathsf{ct}_0 \| (\mathsf{ct}_i)_{i\in[n]}) = \mathsf{h}^*$ and $\mathsf{Inv}(\mathsf{td}, \mathsf{y}) \neq \bot$ hold simultaneously. We categorize a hash-bad inversion query into the following two mutually exclusive types:

- Type-1: $\mathsf{ct}_0 \| (\mathsf{ct}_i)_{i\in[n]} \neq \mathsf{ct}_0^* \| (\mathsf{ct}_i^*)_{i\in[n]}$
- Type-2: $\mathsf{ct}_0 \| (\mathsf{ct}_i)_{i\in[n]} = \mathsf{ct}_0^* \| (\mathsf{ct}_i^*)_{i\in[n]}$

For $t, k \in [2]$, let $\mathtt{HB}_t^k$ be the event that $\mathcal{A}$ makes at least one Type-k hash-bad inversion query in Game t, and let $\mathtt{PKB}_t$ be the event that some of $\{\mathsf{pk}_i\}_{i\in[n]}$ is erroneous in Game t. Observe that if none of $\{\mathsf{pk}_i\}_{i\in[n]}$ is erroneous and $\mathcal{A}$ does not make a hash-bad query, then Game 1 and Game 2 proceed identically. Thus, we have

$$\left|\Pr[\mathtt{T}_1] - \Pr[\mathtt{T}_2]\right| \leq \Pr[\mathtt{PKB}_1 \vee \mathtt{HB}_1^1 \vee \mathtt{HB}_1^2] \leq \Pr[\mathtt{PKB}_1] + \Pr[\mathtt{HB}_1^1] + \Pr[\mathtt{HB}_1^2 | \overline{\mathtt{PKB}_1}].$$

Here, by the definition of the events and the union bound, we have $\Pr[\mathtt{PKB}_1] \leq n \cdot \epsilon$. Furthermore, note that a Type-1 hash-bad query can be directly used to break the target collision resistance of Hash. That is, we can construct a PPT adversary $\mathcal{B}_\mathsf{h}$ with $\Pr[\mathtt{HB}_1^1] = \mathsf{Adv}^{\mathsf{tcr}}_{\mathsf{Hash},\mathcal{B}_\mathsf{h}}(\lambda)$. Since the construction of $\mathcal{B}_\mathsf{h}$ is straightforward, we omit the detail.

It remains to show how $\Pr[\mathtt{HB}_1^2 | \overline{\mathtt{PKB}_1}]$ is bounded. In the following, we show that if none of $\{\mathsf{pk}_i\}_{i\in[n]}$ is erroneous, then except for probability at most $n \cdot 2^{-\lambda}$ over the choice of $\mathsf{C}_1, \ldots, \mathsf{C}_n \xleftarrow{\mathsf{r}} \mathcal{C}$, a Type-2 hash-bad query does not exist in Game 1. To this end, fix all values in Game 1 except for the choice of $\mathsf{C}_1, \ldots, \mathsf{C}_n \in \mathcal{C}$.

Recall that $\mathcal{A}$'s inversion query y must satisfy $\mathsf{y} \neq \mathsf{y}^*$, and thus if $\mathsf{y} = (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$ is a Type-2 hash-bad query (and thus $\mathsf{ct}_0 = \mathsf{ct}_0^*$ and $\mathsf{ct}_i = \mathsf{ct}_i^*$ holds for all $i \in [n]$), then there must exist a position $\alpha \in [n]$ for which $\mathsf{T}_\alpha \neq \mathsf{T}_\alpha^*$ holds. Now, consider an execution of $\mathsf{Inv}(\mathsf{td}, \mathsf{y})$. Observe that by the correctness of decapsulation under a non-erroneous key, for the position $\alpha \in [n]$ with $\mathsf{T}_\alpha \neq \mathsf{T}_\alpha^*$, the check $\mathsf{T}_\alpha = \mathsf{k}_\alpha^{\mathsf{s}_\alpha^*} + \mathsf{s}_\alpha^* \cdot (\mathsf{A}_\alpha + \mathsf{B} \cdot \mathsf{h}^*)$ done in Inv cannot be satisfied, and Inv sets $\mathsf{chk}_\alpha^{\mathsf{s}_\alpha^*} = 0$. Hence, in order for $\mathsf{Inv}(\mathsf{td}, \mathsf{y}) \neq \bot$ to occur, $\mathsf{chk}_\alpha^{1-\mathsf{s}_\alpha^*} = 1$ must be satisfied. That is, we must simultaneously have

- $\mathsf{RDecap}(\mathsf{sk}_\alpha, \mathsf{ct}_\alpha^* - (1 - \mathsf{s}_\alpha^*) \cdot \mathsf{C}_\alpha) = (\mathsf{k}_\alpha^{1-\mathsf{s}_\alpha^*}, \mathsf{r}_\alpha^{1-\mathsf{s}_\alpha^*}) \neq \bot$, and
- $\mathsf{T}_\alpha = \mathsf{k}_\alpha^{1-\mathsf{s}_\alpha^*} + (1 - \mathsf{s}_\alpha^*) \cdot (\mathsf{A}_\alpha + \mathsf{B} \cdot \mathsf{h}^*)$.

In particular, the first condition and the correctness of decapsulation under a non-erroneous key imply that

$$\mathsf{ct}_\alpha^* - (1 - \mathsf{s}_\alpha^*) \cdot \mathsf{C}_\alpha = \mathsf{ct}_\alpha^{*(\mathsf{s}_\alpha^*)} + \mathsf{s}_\alpha^* \cdot \mathsf{C}_\alpha - (1 - \mathsf{s}_\alpha^*) \cdot \mathsf{C}_\alpha = \mathsf{ct}_\alpha^{*(\mathsf{s}_\alpha^*)} - (1 - 2\mathsf{s}_\alpha^*) \cdot \mathsf{C}_\alpha$$

must belong to the image of $\mathsf{Encap}(\mathsf{pk}_\alpha; \cdot)$. Note that the image size of $\mathsf{Encap}(\mathsf{pk}_\alpha; \cdot)$ is at most 2^{ℓ_r}. Since C_α is chosen uniformly at random from $\mathcal{C}$ and $|\mathcal{C}| \geq 2^{\ell_\mathsf{r}+\lambda}$, the probability (over the choice of C_α) that $\mathsf{ct}_\alpha^{*(\mathsf{s}_\alpha^*)} - (1 - 2\mathsf{s}_\alpha^*) \cdot \mathsf{C}_\alpha$ belongs to the image of $\mathsf{Encap}(\mathsf{pk}_\alpha; \cdot)$ is at most $\frac{2^{\ell_\mathsf{r}}}{|\mathcal{C}|} \leq 2^{-\lambda}$. Thus, under the condition that none of $\{\mathsf{pk}_i\}_{i\in[n]}$ is erroneous, except for probability at most $2^{-\lambda}$ over the choice of C_α, a Type-2 hash-bad query y with $\mathsf{T}_\alpha \neq \mathsf{T}_\alpha^*$ does not exist in Game 1. Thus, taking the union bound over all indices in $[n]$, we have $\Pr[\mathtt{HB}_1^2 | \overline{\mathtt{PKB}_1}] \leq n \cdot 2^{-\lambda}$.

Putting everything together, there exists a PPT adversary $\mathcal{B}_\mathsf{h}$ with the claimed advantage, as required. □ (**Lemma 2**)

Lemma 3 $\Pr[\mathtt{T}_2] = \Pr[\mathtt{T}_3]$ *holds.*

Proof of Lemma 3. This is true because the distributions of $\{\mathsf{A}_i\}_{i\in[n]}$ and $\{\mathsf{C}_i\}_{i\in[n]}$ are identical between Games 2 and 3. More specifically, in Game 3, for every $i \in [n]$, $\mathsf{ct}_i^{*(1-\mathsf{s}_i^*)}$ and $\mathsf{k}_i^{*(1-\mathsf{s}_i^*)}$ are chosen uniformly at random and used only for generating C_i and A_i, respectively. These make C_i and A_i distributed uniformly at random, exactly as in Game 2. □ (**Lemma 3**)

$\mathsf{AltInv}(\mathsf{td}, \mathsf{sk}_0, \alpha \in [n], \mathsf{h}^*, \mathsf{y})$:
$((\mathsf{sk}_i)_{i\in[n]}, \mathsf{ek}) \leftarrow \mathsf{td}$ // sk_α can be $\perp$.
$((\mathsf{pk}_i, \mathsf{A}_i, \mathsf{C}_i)_{i\in[n]}, \mathsf{pk}_0, \mathsf{B}, \mathsf{hk}) \leftarrow \mathsf{ek}$
$(\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]}) \leftarrow \mathsf{y}$
$\mathsf{h} \leftarrow \mathsf{H}(\mathsf{hk}, \mathsf{ct}_0 \| (\mathsf{ct}_i)_{i\in[n]})$
If $\mathsf{h} = \mathsf{h}^*$ then return $\perp$.
$\forall (i, v) \in ([n] \setminus \{\alpha\}) \times \{0, 1\}$:
$(\mathsf{k}_i^v, \mathsf{r}_i^v) \leftarrow \mathsf{RDecap}(\mathsf{sk}_i, \mathsf{ct}_i - v \cdot \mathsf{C}_i)$
$\mathsf{chk}_i^v \leftarrow \begin{cases} 1 & \text{if } (\mathsf{k}_i^v, \mathsf{r}_i^v) \neq \perp \wedge \mathsf{T}_i = \mathsf{k}_i^v + v \cdot (\mathsf{A}_i + \mathsf{B} \cdot \mathsf{h}) \\ 0 & \text{otherwise} \end{cases}$
If $\exists i \in [n] \setminus \{\alpha\} : \mathsf{chk}_i^0 = \mathsf{chk}_i^1$ then return $\perp$.
$\forall i \in [n] \setminus \{\alpha\} : \mathsf{s}_i \leftarrow \mathsf{chk}_i^1$
$(\mathsf{k}'_0, \mathsf{r}'_0) \leftarrow \mathsf{RDecap}(\mathsf{sk}_0, \mathsf{ct}_0)$
If $(\mathsf{k}'_0, \mathsf{r}'_0) = \perp$ then return $\perp$.
$\mathsf{r}'_\alpha \leftarrow (\bigoplus_{i\in[n]\setminus\{\alpha\}} \mathsf{r}_i^{\mathsf{s}_i}) \oplus \mathsf{r}'_0$
$(\mathsf{ct}'_\alpha, \mathsf{k}'_\alpha) \leftarrow \mathsf{Encap}(\mathsf{pk}_\alpha; \mathsf{r}'_\alpha)$
$\forall v \in \{0, 1\}$:
$\mathsf{chk}'^v_\alpha \leftarrow \begin{cases} 1 & \text{if } \mathsf{ct}_\alpha = \mathsf{ct}'_\alpha + v \cdot \mathsf{C}_\alpha \wedge \mathsf{T}_\alpha = \mathsf{k}'_\alpha + v \cdot (\mathsf{A}_\alpha + \mathsf{B} \cdot \mathsf{h}) \\ 0 & \text{otherwise} \end{cases}$
If $\mathsf{chk}'^0_\alpha = \mathsf{chk}'^1_\alpha$ then return $\perp$.
$(\mathsf{s}_\alpha, \mathsf{r}_\alpha^{\mathsf{s}_\alpha}) \leftarrow (\mathsf{chk}'^1_\alpha, \mathsf{r}'_\alpha)$
Return $\mathsf{x} \leftarrow (\mathsf{s} = (\mathsf{s}_1, \ldots, \mathsf{s}_n), (\mathsf{r}_i^{\mathsf{s}_i})_{i\in[n]})$.

Figure 5: The alternative inversion algorithm AltInv. We use the same notation as in Figure 4.

Lemma 4 *There exists a PPT adversary $\mathcal{B}$ such that*

$$\left|\Pr[\mathtt{T}_3] - \Pr[\mathtt{T}_4]\right| \leq n \cdot \mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM},\mathcal{B}}(\lambda) + 2n(n+1)\epsilon + 2n^2 \cdot 2^{-\lambda}.$$

Proof of Lemma 4. In the proof of this lemma, we will use the alternative decapsulation algorithm AltInv that takes $\mathsf{td} = ((\mathsf{sk}_i)_{i\in[n]}, \mathsf{ek})$, sk_0, $\alpha \in [n]$, $\mathsf{h}^* \in \{0,1\}^\lambda$, and an instance $\mathsf{y} = (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$ as input, and inverts y *without using* sk_α. Specifically, the description of AltInv is given in Figure 5.

We introduce the following intermediate games between Games 3 and 4.

Game $3.j$**:** (for $j \in \{0\} \cup [n]$) Same as Game 3, except that for $i \leq j$, $(\mathsf{ct}_i^{*(1-\mathsf{s}_i^*)}, \mathsf{k}_i^{*(1-\mathsf{s}_i^*)})$ is generated as $(\mathsf{ct}_i^{*(1-\mathsf{s}_i^*)}, \mathsf{k}_i^{*(1-\mathsf{s}_i^*)}) \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{*(1-\mathsf{s}_i^*)})$, instead of being picked randomly from $\mathcal{C} \times \{0,1\}^{4\lambda}$.

Note that in this game, the inversion oracle responds to a query $\mathsf{y} = (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$ with $\mathsf{Inv}(\mathsf{td}, \mathsf{y})$, where we also have the rejection rule for rejecting a query with $\mathsf{H}(\mathsf{hk}, \mathsf{ct}_0 \| (\mathsf{ct}_i)_{i\in[n]}) = \mathsf{h}^*$ introduced in Game 2. Hence, Game 3 (resp. Game 4) is identical to Game 3.0 (resp. Game $3.n$).

Game $3.j.a$**:** (for $j \in [n]$) Same as Game $3.(j-1)$, except that the inversion oracle responds to a query y with $\mathsf{AltInv}(\mathsf{td}, \mathsf{sk}_0, j, \mathsf{h}^*, \mathsf{y})$.

Game $3.j.b$**:** (for $j \in [n]$) Same as Game $3.j.a$, except that for $i \leq j$, $(\mathsf{ct}_i^{*(1-\mathsf{s}_i^*)}, \mathsf{k}_i^{*(1-\mathsf{s}_i^*)})$ is generated as $(\mathsf{ct}_i^{*(1-\mathsf{s}_i^*)}, \mathsf{k}_i^{*(1-\mathsf{s}_i^*)}) \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{*(1-\mathsf{s}_i^*)})$, instead of being picked randomly from $\mathcal{C} \times \{0,1\}^{4\lambda}$.

Table 1: An overview of the sequence of games between Game $3.(j-1)$ and $3.j$ for $j \in [n]$. $^{(\dagger)}$ The inversion oracle also has the rejection rule for rejecting a query with $\mathsf{H}(\mathsf{hk}, \mathsf{ct}_0 \| (\mathsf{ct}_i)_{i\in[n]}) = \mathsf{h}^*$ introduced in Game 2.

Game	$(\mathsf{ct}_i^{*(1-\mathsf{s}_i^*)}, \mathsf{k}_i^{*(1-\mathsf{s}_i^*)})$	Inversion oracle
$3.(j-1)$	$\begin{cases} \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{*(1-\mathsf{s}_i^*)}) & \text{if } i \le j-1 \\ \xleftarrow{\mathsf{r}} \mathcal{C} \times \{0,1\}^{4\lambda} & \text{if } i \ge j \end{cases}$	$\mathsf{Inv}(\mathsf{td}, \cdot)$ $^{(\dagger)}$
$3.j.a$	$\begin{cases} \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{*(1-\mathsf{s}_i^*)}) & \text{if } i \le j-1 \\ \xleftarrow{\mathsf{r}} \mathcal{C} \times \{0,1\}^{4\lambda} & \text{if } i \ge j \end{cases}$	$\mathsf{AltInv}(\mathsf{td}, \mathsf{sk}_0, j, \mathsf{h}^*, \cdot)$
$3.j.b$	$\begin{cases} \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{*(1-\mathsf{s}_i^*)}) & \text{if } i \le j \\ \xleftarrow{\mathsf{r}} \mathcal{C} \times \{0,1\}^{4\lambda} & \text{if } i \ge j+1 \end{cases}$	$\mathsf{AltInv}(\mathsf{td}, \mathsf{sk}_0, j, \mathsf{h}^*, \cdot)$
$3.j$	$\begin{cases} \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{*(1-\mathsf{s}_i^*)}) & \text{if } i \le j \\ \xleftarrow{\mathsf{r}} \mathcal{C} \times \{0,1\}^{4\lambda} & \text{if } i \ge j+1 \end{cases}$	$\mathsf{Inv}(\mathsf{td}, \cdot)$ $^{(\dagger)}$

Note that the difference between this game and Game $3.j$ is only in the inversion oracle: AltInv is used in Game $3.j.b$, while Inv (with the rejection rule regarding h^* as in Game 2) is used in Game $3.j$.

We will consider transitions of the games in the following order: $3.0 \to 3.1.a \to 3.1.b \to 3.1 \to 3.2.a \to \cdots \to 3.n.b \to 3.n$. See Table 1 for an overview of the sequence.

In these intermediate games, for $\alpha \in [n]$, we call an inversion query $\mathsf{y} = (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$ α-*bad* if it satisfies $\mathsf{H}(\mathsf{hk}, \mathsf{ct}_0 \| (\mathsf{ct}_i)_{i\in[n]}) \neq \mathsf{h}^*$ and $\mathsf{Inv}(\mathsf{td}, \mathsf{y}) \neq \mathsf{AltInv}(\mathsf{td}, \mathsf{sk}_0, \alpha, \mathsf{h}^*, \mathsf{y})$ simultaneously. For Game t and $\alpha \in [n]$, we denote by $\mathtt{BQ}_t^\alpha$ the event that $\mathcal{A}$ submits an α-bad inversion query in Game t. Also, we continue to denote by $\mathtt{T}_t$ the event that $\mathcal{A}$ outputs $b' = 1$ in Game t.

Observe that for $j \in [n]$, Game $3.(j-1)$ and Game $3.j.a$ proceed identically unless $\mathcal{A}$ submits a j-bad inversion query. Thus, we have $|\Pr[\mathtt{T}_{3.(j-1)}] - \Pr[\mathtt{T}_{3.j.a}]| \le \Pr[\mathtt{BQ}^j_{3.(j-1)}]$. Similarly, Game $3.j.b$ and Game $3.j$ proceed identically unless $\mathcal{A}$ submits a j-bad inversion query. Thus, we have $|\Pr[\mathtt{T}_{3.j.b}] - \Pr[\mathtt{T}_{3.j}]| \le \Pr[\mathtt{BQ}^j_{3.j}]$.

Using them, we can estimate $|\Pr[\mathtt{T}_3] - \Pr[\mathtt{T}_4]|$ as follows.

$$
\begin{aligned}
&\Big|\Pr[\mathtt{T}_3] - \Pr[\mathtt{T}_4]\Big| = \Big|\Pr[\mathtt{T}_{3.0}] - \Pr[\mathtt{T}_{3.n}]\Big| = \Big|\sum_{j\in[n]} \Big(\Pr[\mathtt{T}_{3.(j-1)}] - \Pr[\mathtt{T}_{3.j}]\Big)\Big| \\
&= \Big|\sum_{j\in[n]} \Big(\Pr[\mathtt{T}_{3.(j-1)}] - \Pr[\mathtt{T}_{3.j.a}] + \Pr[\mathtt{T}_{3.j.a}] - \Pr[\mathtt{T}_{3.j.b}] + \Pr[\mathtt{T}_{3.j.b}] - \Pr[\mathtt{T}_{3.j}]\Big)\Big| \\
&\le \sum_{j\in[n]} \Big|\Pr[\mathtt{T}_{3.(j-1)}] - \Pr[\mathtt{T}_{3.j.a}]\Big| + \sum_{j\in[n]} \Big|\Pr[\mathtt{T}_{3.j.b}] - \Pr[\mathtt{T}_{3.j}]\Big| \\
&\quad + \Big|\sum_{j\in[n]} \Big(\Pr[\mathtt{T}_{3.j.a}] - \Pr[\mathtt{T}_{3.j.b}]\Big)\Big| \\
&\le \sum_{j\in[n]} \Pr[\mathtt{BQ}^j_{3.(j-1)}] + \sum_{j\in[n]} \Pr[\mathtt{BQ}^j_{3.j}] + \Big|\sum_{j\in[n]} \Big(\Pr[\mathtt{T}_{3.j.a}] - \Pr[\mathtt{T}_{3.j.b}]\Big)\Big|
\end{aligned}
$$

We will show the following claims which, combined with the above inequality, imply the lemma.

Claim 1 *For all $j \in \{0\} \cup [n]$ and $\alpha \in [n]$, we have $\Pr[\mathtt{BQ}^\alpha_{3.j}] \le (n+1) \cdot \epsilon + n \cdot 2^{-\lambda}$.*

Proof of Claim 1. Fix all values of ek so that pk_i's are all non-erroneous, td, and y^*, except for B. For each $i \in [n]$, define a function $g_i : \{0,1\}^{\ell_\mathsf{r}} \times (\{0,1\}^\lambda \setminus \{\mathsf{h}^*\}) \to \{0,1\}^{4\lambda} \cup \{\bot\}$ by

$$g_i(\mathsf{r}, \mathsf{h}) : \left[\begin{array}{l} (\mathsf{ct}, \mathsf{k}) \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r});\ (\mathsf{k}', \mathsf{r}') \leftarrow \mathsf{RDecap}(\mathsf{sk}_i, \mathsf{ct} - \mathsf{C}_i); \\ \text{If } (\mathsf{k}', \mathsf{r}') = \bot \text{ then return } \bot \text{ else return } \frac{\mathsf{k} - \mathsf{k}' - \mathsf{k}_i^{*0} + \mathsf{k}_i^{*1}}{\mathsf{h} - \mathsf{h}^*} \end{array} \right].$$

We say that an evaluation key/trapdoor pair $(\mathsf{ek}, \mathsf{td})$ is *bad* if (1) some of $\{\mathsf{pk}_i\}_{i \in \{0\} \cup [n]}$ is erroneous, or (2) B belongs to the image of g_i for some $i \in [n]$. Otherwise, we call $(\mathsf{ek}, \mathsf{td})$ *good*. Since the number of possible session-keys output by Encap (for any public key output by KKG) is assumed to be at most 2^λ, k and k' can each take at most 2^λ values. Hence, the image size of each g_i (excluding $\bot$) is at most $2^{3\lambda}$. Since B is chosen uniformly at random from $\{0,1\}^{4\lambda}$ in Games 3 and 4 and all the intermediate games, the probability that a bad pair $(\mathsf{ek}, \mathsf{td})$ is used in each of these games is at most $(n+1) \cdot \epsilon + n \cdot \frac{2^{3\lambda}}{2^{4\lambda}} = (n+1)\epsilon + n \cdot 2^{-\lambda}$ by the union bound.

Fix arbitrarily $\alpha \in [n]$. In the rest of the proof, we show that if $(\mathsf{ek}, \mathsf{td})$ is good, then an α-bad inversion query does not exist in Games 3 and 4 and all the intermediate games. This, combined with the above argument on the probability of $(\mathsf{ek}, \mathsf{td})$ being bad, will complete the proof of the claim.

Let $\mathsf{y} = (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i \in [n]})$ be any inversion query satisfying $\mathsf{H}(\mathsf{hk}, \mathsf{ct}_0 \| (\mathsf{ct}_i)_{i \in [n]}) \neq \mathsf{h}^*$. Let $\mathsf{x} := \mathsf{Inv}(\mathsf{td}, \mathsf{y})$ and $\mathsf{x}' := \mathsf{AltInv}(\mathsf{td}, \mathsf{sk}_0, j, \mathsf{h}^*, \mathsf{y})$. For each $(i, v) \in [n] \times \{0,1\}$, let $(\mathsf{k}_i^v, \mathsf{r}_i^v) := \mathsf{RDecap}(\mathsf{sk}_i, \mathsf{ct}_i - v \cdot \mathsf{C}_i)$. Let $(\mathsf{k}_0', \mathsf{r}_0') := \mathsf{RDecap}(\mathsf{sk}_0, \mathsf{ct}_0)$.[13]

Below, we consider four cases based on the condition satisfied by y, and for each case, we show that $\mathsf{x} = \mathsf{x}'$ holds.

- Case 1: There exists $i \in [n] \setminus \{\alpha\}$ such that $\mathsf{chk}_i^0 = \mathsf{chk}_i^1$. In this case, both Inv and AltInv output $\bot$ by design. (These values are computed identically in both Inv and AltInv.)

- Case 2: $(\mathsf{k}_0', \mathsf{r}_0') = \bot$. In this case, AltInv outputs $\bot$ by design. Inv also outputs $\bot$ since the condition of this case and the correctness of KEM under a non-erroneous key imply that there does not exist r satisfying $\mathsf{Encap}(\mathsf{pk}_0; \mathsf{r}) = (\mathsf{ct}_0, *)$, and thus the validity check of ct_0 performed at the second last step of Inv is never satisfied.

- Case 3: For both $v \in \{0,1\}$, either $(\mathsf{k}_\alpha^v, \mathsf{r}_\alpha^v) = \bot$ or $\mathsf{T}_\alpha \neq \mathsf{k}_\alpha^v + v \cdot (\mathsf{A}_\alpha + \mathsf{B} \cdot \mathsf{h})$ holds. In this case, Inv sets $\mathsf{chk}_\alpha^v \leftarrow 0$ for both $v \in \{0,1\}$, and thus outputs $\bot$ when it checks whether chk_α^0 and chk_α^1 are equal.

  Moreover, the condition of this case and the correctness of KEM under a non-erroneous key imply that there does not exist a pair $(\mathsf{r}, v) \in \{0,1\}^{\ell_\mathsf{r}} \times \{0,1\}$ that satisfies $\mathsf{Encap}(\mathsf{pk}_\alpha; \mathsf{r}) = (\mathsf{ct}_\alpha - v \cdot \mathsf{C}_\alpha, \mathsf{T}_\alpha - v \cdot (\mathsf{A}_\alpha + \mathsf{B} \cdot \mathsf{h}))$. Thus, AltInv also sets $\mathsf{chk}_\alpha'^v \leftarrow 0$ for both $v \in \{0,1\}$, and thus AltInv outputs $\bot$ as well when it checks whether $\mathsf{chk}_\alpha'^0$ and $\mathsf{chk}_\alpha'^1$ are equal.

- Case 4: None of the conditions of Cases 1 to 3 hold. In this case, by the negated condition of Case 3, there exists $\beta \in \{0,1\}$ such that the following hold simultaneously:

$$(\mathsf{k}_\alpha^\beta, \mathsf{r}_\alpha^\beta) \neq \bot \quad \text{and} \quad \mathsf{T}_\alpha = \mathsf{k}_\alpha^\beta + \beta \cdot (\mathsf{A}_\alpha + \mathsf{B} \cdot \mathsf{h}). \tag{8}$$

  We argue that the following condition is also satisfied:

$$(\mathsf{k}_\alpha^{1-\beta}, \mathsf{r}_\alpha^{1-\beta}) = \bot \quad \text{or} \quad \mathsf{T}_\alpha \neq \mathsf{k}_\alpha^{1-\beta} + (1-\beta) \cdot (\mathsf{A}_\alpha + \mathsf{B} \cdot \mathsf{h}). \tag{9}$$

  To see this, assume towards a contradiction that the negation of Eq. (9) is satisfied. Then, combining the equalities regarding T_α, we have

$$\mathsf{k}_\alpha^\beta + \beta \cdot (\mathsf{A}_\alpha + \mathsf{B} \cdot \mathsf{h}) = \mathsf{k}_\alpha^{1-\beta} + (1-\beta) \cdot (\mathsf{A}_\alpha + \mathsf{B} \cdot \mathsf{h}) \iff \mathsf{k}_\alpha^0 = \mathsf{k}_\alpha^1 + \mathsf{A}_\alpha + \mathsf{B} \cdot \mathsf{h}. \tag{10}$$

[13] Note that $(\mathsf{k}_i^v, \mathsf{r}_i^v)$ and/or $(\mathsf{k}_0', \mathsf{r}_0')$ can be $\bot$ in case the decapsulated ciphertext is invalid.

On the other hand, since B is not in the image of g_α, we have

$$\mathsf{B} \neq g_\alpha(\mathsf{r}_\alpha^0, \mathsf{h}) = \frac{\mathsf{k}_\alpha^0 - \mathsf{k}_\alpha^1 - \mathsf{k}_\alpha^{*0} + \mathsf{k}_\alpha^{*1}}{\mathsf{h} - \mathsf{h}^*}$$

$$\iff \quad \mathsf{k}_\alpha^0 \neq \mathsf{k}_\alpha^1 + (\mathsf{k}_\alpha^{*0} - \mathsf{k}_\alpha^{*1} - \mathsf{B} \cdot \mathsf{h}^*) + \mathsf{B} \cdot \mathsf{h} \overset{(*)}{=} \mathsf{k}_\alpha^1 + \mathsf{A}_\alpha + \mathsf{B} \cdot \mathsf{h}, \tag{11}$$

where the equality (*) uses $\mathsf{A}_\alpha = \mathsf{k}_\alpha^{*0} - \mathsf{k}_\alpha^{*1} - \mathsf{B} \cdot \mathsf{h}^*$ which is how A_α is defined in Games 3 and 4 and all the intermediate games. However, Eqs. (10) and (11) contradict each other. Thus, Eq. (9) must hold.

Eqs. (8) and (9) imply that Inv sets $\mathsf{chk}_\alpha^\beta \leftarrow 1$ and $\mathsf{chk}_\alpha^{1-\beta} \leftarrow 0$, respectively. Furthermore, these equalities, together with the correctness of KEM under a non-erroneous key, imply the following:

$$\mathsf{Encap}(\mathsf{pk}_\alpha; \mathsf{r}_\alpha^\beta) = (\mathsf{ct}_\alpha - \beta \cdot \mathsf{C}_\alpha, \mathsf{T}_\alpha - \beta \cdot (\mathsf{A}_\alpha + \mathsf{B} \cdot \mathsf{h})), \tag{12}$$

$$\forall \mathsf{r} \in \{0,1\}^{\ell_r} : \quad \mathsf{Encap}(\mathsf{pk}_\alpha; \mathsf{r}) \neq (\mathsf{ct}_\alpha - (1-\beta) \cdot \mathsf{C}_\alpha, \mathsf{T}_\alpha - (1-\beta) \cdot (\mathsf{A}_\alpha + \mathsf{B} \cdot \mathsf{h})). \tag{13}$$

The negated condition of Case 1 and the above argument on $\mathsf{chk}_\alpha^\beta$ and $\mathsf{chk}_\alpha^{1-\beta}$ imply that $\mathsf{chk}_i^0 \neq \mathsf{chk}_i^1$ holds for all $i \in [n]$ in an execution of Inv. For each $i \in [n]$, let $\mathsf{s}_i := \mathsf{chk}_i^1$ (as computed in Inv). Note that $(\mathsf{chk}_\alpha^\beta, \mathsf{chk}_\alpha^{1-\beta}) = (1,0) \Leftrightarrow (\mathsf{chk}_\alpha^0, \mathsf{chk}_\alpha^1) = (1-\beta, \beta)$, and thus $\mathsf{s}_\alpha = \mathsf{chk}_\alpha^1 = \beta$ holds.

Let $\mathsf{r}_0 := \bigoplus_{i \in [n]} \mathsf{r}_i^{\mathsf{s}_i}$ and $\mathsf{r}'_\alpha := \bigoplus_{i \in [n] \setminus \{\alpha\}} \mathsf{r}_i^{\mathsf{s}_i} \oplus \mathsf{r}'_0$. Note that we have

$$\mathsf{r}'_\alpha = \mathsf{r}_\alpha^{\mathsf{s}_\alpha} \oplus \mathsf{r}_0 \oplus \mathsf{r}'_0 = \mathsf{r}_\alpha^\beta \oplus \mathsf{r}_0 \oplus \mathsf{r}'_0. \tag{14}$$

Eq. (13) implies that AltInv sets $\mathsf{chk}'^{1-\beta}_\alpha \leftarrow 0$. Thus, how $\mathsf{chk}'^\beta_\alpha$ is determined in AltInv directly affects its output. We consider two sub-cases based on whether $\mathsf{r}_0 = \mathsf{r}'_0$ or not.

- Sub-case $\mathsf{r}_0 \neq \mathsf{r}'_0$. Since $\mathsf{RDecap}(\mathsf{sk}_0, \mathsf{ct}_0) = (*, \mathsf{r}'_0) \neq \bot$ holds, $\mathsf{r}_0 \neq \mathsf{r}'_0$ and the correctness of KEM under a non-erroneous key imply that $\mathsf{Encap}(\mathsf{pk}_0; \mathsf{r}_0)$ never outputs ct_0. Thus, Inv outputs $\bot$.

  On the other hand, $\mathsf{r}_0 \neq \mathsf{r}'_0$ and Eq. (14) imply $\mathsf{r}'_\alpha \neq \mathsf{r}_\alpha^\beta$. Since $\mathsf{Encap}(\mathsf{pk}_\alpha; \cdot)$ is injective (due to the correctness of KEM under a non-erroneous key), $\mathsf{r}'_\alpha \neq \mathsf{r}_\alpha^\beta$ and Eq. (12) imply $\mathsf{Encap}(\mathsf{pk}_\alpha; \mathsf{r}'_\alpha) \neq (\mathsf{ct}_\alpha - \beta \cdot \mathsf{C}_\alpha, \mathsf{T}_\alpha - \beta \cdot (\mathsf{A}_\alpha + \mathsf{B} \cdot \mathsf{h}))$. This in turn makes AltInv set $\mathsf{chk}'^\beta_\alpha \leftarrow 0$. Hence, AltInv outputs $\bot$ as well when it checks whether chk'^0_α and chk'^1_α are equal.
- Sub-case $\mathsf{r}_0 = \mathsf{r}'_0$. The condition of this sub-case and the negated condition of Case 2 imply $\mathsf{Encap}(\mathsf{pk}_0; \mathsf{r}_0) = \mathsf{Encap}(\mathsf{pk}_0; \mathsf{r}'_0) = (\mathsf{ct}_0, *)$, and thus Inv outputs $\mathsf{x} = (\mathsf{s} = (\mathsf{s}_1, \ldots, \mathsf{s}_n), (\mathsf{r}_i^{\mathsf{s}_i})_{i \in [n]})$.

  On the other hand, $\mathsf{r}_0 = \mathsf{r}'_0$ and Eq. (14) imply $\mathsf{r}'_\alpha = \mathsf{r}_\alpha^\beta$. Then, due to Eq. (12), AltInv sets $\mathsf{chk}'^\beta_1 \leftarrow 1$. Since $(\mathsf{chk}'^\beta_\alpha, \mathsf{chk}'^{1-\beta}_\alpha) = (1,0) \Leftrightarrow (\mathsf{chk}'^0_\alpha, \mathsf{chk}'^1_\alpha) = (1-\beta, \beta)$, AltInv sets $(\mathsf{s}_\alpha, \mathsf{r}_\alpha^{\mathsf{s}_\alpha}) \leftarrow (\mathsf{chk}'^1_\alpha, \mathsf{r}'_\alpha) = (\beta, \mathsf{r}_\alpha^\beta)$ in its second last step, and these values are identical to $(\mathsf{s}_\alpha, \mathsf{r}_\alpha^{\mathsf{s}_\alpha})$ in Inv. Since $\{(\mathsf{s}_i, \mathsf{r}_i^{\mathsf{s}_i})\}_{i \in [n] \setminus \{\alpha\}}$ are also computed identically in both Inv and AltInv, the output x' of AltInv is identical to $\mathsf{x} = (\mathsf{s} = (\mathsf{s}_1, \ldots, \mathsf{s}_n), (\mathsf{r}_i^{\mathsf{s}_i})_{i \in [n]})$.

We have seen that if $(\mathsf{ek}, \mathsf{td})$ is good, then $\mathsf{x} = \mathsf{x}'$ holds in all cases and thus there does not exist an α-bad inversion query, as required. □ (**Claim 1**)

Claim 2 *There exists a PPT adversary $\mathcal{B}$ such that*

$$\Big|\sum_{j\in[n]}\Big(\Pr[\mathtt{T}_{3.j.a}]-\Pr[\mathtt{T}_{3.j.b}]\Big)\Big| = n\cdot \mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM},\mathcal{B}}(\lambda).$$

Proof of Claim 2. Using $\mathcal{A}$ as a building block, we show how to construct a PPT adversary $\mathcal{B}$ that attacks the pseudorandom ciphertext property of KEM with the claimed advantage. The description of $\mathcal{B}$ is as follows.

$\mathcal{B}(\mathsf{pk}', \mathsf{ct}'^*_\beta, \mathsf{k}'^*_\beta)$**:** (where $\beta\in\{0,1\}$ denotes $\mathcal{B}$'s challenge bit) First, $\mathcal{B}$ picks $u \xleftarrow{\mathsf{r}} [n]$ uniformly at random, and prepares ek and y^* as follows.

1. Pick $\mathsf{s}^* = (\mathsf{s}^*_1,\ldots,\mathsf{s}^*_n) \xleftarrow{\mathsf{r}} \{0,1\}^n$ and $\mathsf{r}^{*v}_i \xleftarrow{\mathsf{r}} \{0,1\}^{\ell_\mathsf{r}}$ for every $(i,v)\in[n]\times\{0,1\}\setminus\{(u,1-\mathsf{s}^*_u)\}$. Also, pick $\mathsf{B}\xleftarrow{\mathsf{r}}\{0,1\}^{4\lambda}$.
2. Set $(\mathsf{pk}_u,\mathsf{sk}_u)\leftarrow(\mathsf{pk}',\bot)$, and compute $(\mathsf{pk}_i,\mathsf{sk}_i)\leftarrow\mathsf{KKG}(1^\lambda)$ for every $i\in\{0,\ldots,n\}\setminus\{u\}$.
3. Compute $(\mathsf{ct}^{*(\mathsf{s}^*_i)}_i, \mathsf{k}^{*(\mathsf{s}^*_i)}_i)\leftarrow\mathsf{Encap}(\mathsf{pk}_i;\mathsf{r}^{*(\mathsf{s}^*_i)}_i)$ for every $i\in[n]$.
4. For every $i\in[n]$, compute/set
$$(\mathsf{ct}^{*(1-\mathsf{s}^*_i)}_i, \mathsf{k}^{*(1-\mathsf{s}^*_i)}_i)\begin{cases}\leftarrow\mathsf{Encap}(\mathsf{pk}_i;\mathsf{r}^{*(1-\mathsf{s}^*_i)}_i) & \text{if } i\le u-1\\ \leftarrow(\mathsf{ct}'^*_\beta,\mathsf{k}'^*_\beta) & \text{if } i=u\\ \xleftarrow{\mathsf{r}}\mathcal{C}\times\{0,1\}^{4\lambda} & \text{if } i\ge u+1\end{cases}.$$
5. Set $\mathsf{ct}^*_i\leftarrow\mathsf{ct}^{*0}_i$ and $\mathsf{C}_i\leftarrow\mathsf{ct}^{*0}_i-\mathsf{ct}^{*1}_i$ for every $i\in[n]$.
6. Compute $\mathsf{r}^*_0\leftarrow\bigoplus_{i\in[n]}\mathsf{r}^{*(\mathsf{s}^*_i)}_i$ and $(\mathsf{ct}^*_0,\mathsf{k}^*_0)\leftarrow\mathsf{Encap}(\mathsf{pk}_0;\mathsf{r}^*_0)$.
7. Compute $\mathsf{hk}\leftarrow\mathsf{HKG}(1^\lambda)$ and $\mathsf{h}^*\leftarrow\mathsf{H}(\mathsf{hk},\mathsf{ct}^*_0\|(\mathsf{ct}^*_i)_{i\in[n]})$.
8. Compute $\mathsf{A}_i\leftarrow\mathsf{k}^{*0}_i-\mathsf{k}^{*1}_i-\mathsf{B}\cdot\mathsf{h}^*$ and $\mathsf{T}^*_i\leftarrow\mathsf{k}^{*0}_i$ for every $i\in[n]$.
9. Set $\mathsf{ek}\leftarrow((\mathsf{pk}_i,\mathsf{A}_i,\mathsf{C}_i)_{i\in[n]},\mathsf{pk}_0,\mathsf{B},\mathsf{hk})$, $\mathsf{td}\leftarrow((\mathsf{sk}_i)_{i\in[n]},\mathsf{ek})$, and $\mathsf{y}^*\leftarrow(\mathsf{ct}^*_0,(\mathsf{ct}^*_i,\mathsf{T}^*_i)_{i\in[n]})$.

Then, $\mathcal{B}$ runs $\mathcal{A}(\mathsf{ek},\mathsf{y}^*)$. From here on, $\mathcal{A}$ may start making inversion queries $\mathsf{y}=(\mathsf{ct}_0,(\mathsf{ct}_i,\mathsf{T}_i)_{i\in[n]})$, which are answered with $\mathsf{x}\leftarrow\mathsf{AltInv}(\mathsf{td},\mathsf{sk}_0,u,\mathsf{h}^*,\mathsf{y})$. (Note that to execute AltInv here, sk_u corresponding to pk_u is not needed.)

When $\mathcal{A}$ terminates with output b', $\mathcal{B}$ sets $\beta'\leftarrow b'$, and terminates with output β'.

It is not hard to see that if $\mathcal{B}$'s challenge bit β is 0 (resp. 1), $\mathcal{B}$ perfectly simulates Game $3.u.a$ (resp. Game $3.u.b$) for $\mathcal{A}$. Under this situation, for every $j\in[n]$, if $u=j$, then the probability that $\mathcal{A}$ outputs $b'=1$ (and thus $\mathcal{B}$ also outputs $\beta'=1$) is exactly $\Pr[\mathtt{T}_{3.j.a}]$, i.e. we have $\Pr[\beta'=1|\beta=0\wedge u=j]=\Pr[\mathtt{T}_{3.j.a}]$. Since $u\in[n]$ is chosen uniformly at random, independently of β, we have $\Pr[u=j|\beta=0]=\Pr[u=j|\beta=1]=1/n$ for every $j\in[n]$. Hence, we have

$$\Pr[\beta'=1|\beta=0]=\frac{1}{n}\sum_{j\in[n]}\Pr[\beta'=1|\beta=0\wedge u=j]=\frac{1}{n}\sum_{j\in[n]}\Pr[\mathtt{T}_{3.j.a}].$$

Similarly, we have $\Pr[\beta'=1|\beta=1\wedge u=j]=(1/n)\cdot\sum_{j\in[n]}\Pr[\mathtt{T}_{3.j.b}]$. Hence, $\mathcal{B}$'s advantage can be calculated as follows:

$$\mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM},\mathcal{B}}(\lambda)=\Big|\Pr[\beta'=1|\beta=0]-\Pr[\beta'=1|\beta=1]\Big|=\frac{1}{n}\Big|\sum_{j\in[n]}\Big(\Pr[\mathtt{T}_{3.j.a}]-\Pr[\mathtt{T}_{3.j.b}]\Big)\Big|,$$

which is equivalent to the claimed equality. □ (**Claim 2**)

□ (**Lemma 4**)

Lemma 5 $|\Pr[\mathtt{T}_4] - \Pr[\mathtt{T}_5]| \leq 2^{-\lambda-1}$ *holds.*

Proof of Lemma 5. For a $2 \times n$ matrix $\mathsf{R} = (\mathsf{r}_i^v)_{i\in[n],v\in\{0,1\}} \in (\{0,1\}^{\ell_\mathsf{r}})^{2\times n}$, define a function $H_\mathsf{R} : \{0,1\}^n \to \{0,1\}^{\ell_\mathsf{r}}$ by $H_\mathsf{R}(\mathsf{s} = (\mathsf{s}_1, \ldots, \mathsf{s}_n)) = \bigoplus_{i\in[n]} \mathsf{r}_i^{\mathsf{s}_i}$. Then, $\mathcal{H} := \{H_\mathsf{R} : \{0,1\}^n \to \{0,1\}^{\ell_\mathsf{r}} | \mathsf{R} \in (\{0,1\}^{\ell_\mathsf{r}})^{2\times n}\}$ constitutes a family of universal hash functions. Note that r_0^* in Game 4 can be seen as computed as $\mathsf{r}_0^* = H_{\mathsf{R}^*}(\mathsf{s}^*)$, where $\mathsf{R}^* := (\mathsf{r}_i^{*v})_{i\in[n],v\in\{0,1\}}$ are the randomnesses behind $\{\mathsf{ct}_i^{*v}\}_{i\in[n],v\in\{0,1\}}$. Note also that in Game 4, s^* is used only to compute r_0^*, and the latter is replaced with a uniformly random string in Game 5. Hence, by the leftover hash lemma (Lemma 1), we have $|\Pr[\mathtt{T}_4] - \Pr[\mathtt{T}_5]| \leq (1/2) \cdot \sqrt{2^{\ell_\mathsf{r}-n}} = 2^{-\lambda-1}$, where the last equality uses $n = \ell_\mathsf{r} + 2\lambda$. □ (**Lemma 5**)

Lemma 6 *There exists a PPT adversary* $\mathcal{B}'$ *such that* $|\Pr[\mathtt{T}_5] - \Pr[\mathtt{T}_6]| = \mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM},\mathcal{B}'}(\lambda)$.

Proof of Lemma 6. Note that in Game 5, the randomness r_0^* behind ct_0^* is chosen uniformly at random. Furthermore, the secret key sk_0 is not used in the trapdoor td. Hence, we can straightforwardly construct a reduction algorithm $\mathcal{B}'$ that embeds its challenge instance into the position of $\mathsf{pk}_0/\mathsf{ct}_0^*$, simulates Game 5 or 6 for $\mathcal{A}$ depending on the challenge bit of $\mathcal{B}'$, and has the claimed advantage. (The reduction is straightforward and thus omitted.) □ (**Lemma 6**)

Lemma 7 *There exists a PPT adversary* $\mathcal{B}''$ *such that*

$$\left|\Pr[\mathtt{T}_6] - \Pr[\mathtt{T}_7]\right| \leq 2n \cdot \mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM},\mathcal{B}''}(\lambda) + 2n(n+1)\epsilon + 2n^2 \cdot 2^{-\lambda}.$$

Proof of Lemma 7. The proof of this lemma is analogous to the proof of Lemma 4. We introduce the following intermediate games between Games 6 and 7.

Game $6.j$**:** (for $j \in \{0\} \cup [n]$) Same as Game 6, except that for $i \leq j$, $(\mathsf{ct}_i^{*v}, \mathsf{k}_i^{*v})$ for both $v \in \{0,1\}$ is chosen uniformly at random from $\mathcal{C} \times \{0,1\}^{4\lambda}$, instead of being generated as $(\mathsf{ct}_i^{*v}, \mathsf{k}_i^{*v}) \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{*v})$.

Note that in this game, the inversion oracle responds to a query $\mathsf{y} = (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$ with $\mathsf{Inv}(\mathsf{td}, \mathsf{y})$, where we also have the rejection rule for rejecting a query with $\mathsf{H}(\mathsf{hk}, \mathsf{ct}_0 \| (\mathsf{ct}_i)_{i\in[n]}) = \mathsf{h}^*$ introduced in Game 2. Hence, Game 6 (resp. Game 7) is identical to Game 6.0 (resp. Game $6.n$).

Game $6.j.a$**:** (for $j \in [n]$) Same as Game $6.(j-1)$, except that the inversion oracle responds to a query y with $\mathsf{AltInv}(\mathsf{td}, \mathsf{sk}_0, j, \mathsf{h}^*, \mathsf{y})$.

Game $6.j.b$**:** (for $j \in [n]$) Same as Game $6.j.a$, except that for $i \leq j$, $(\mathsf{ct}_i^{*v}, \mathsf{k}_i^{*v})$ for both $v \in \{0,1\}$ is chosen uniformly at random from $\mathcal{C} \times \{0,1\}^{4\lambda}$, instead of being generated as $(\mathsf{ct}_i^{*v}, \mathsf{k}_i^{*v}) \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{*v})$.

Note that the difference between this game and Game $6.j$ is only in the inversion oracle: AltInv is used in Game $6.j.b$, while Inv (with the rejection rule regarding h^* as in Game 2) is used in Game $6.j$.

Table 2: An overview of the sequence of games between Game $6.(j-1)$ and $6.j$ for $j \in [n]$. $^{(\dagger)}$ The inversion oracle also has the rejection rule for rejecting a query with $\mathsf{H}(\mathsf{hk}, \mathsf{ct}_0 \| (\mathsf{ct}_i)_{i\in[n]}) = \mathsf{h}^*$ introduced in Game 2.

Game	$(\mathsf{ct}_i^{*v}, \mathsf{k}_i^{*v})$ for $v \in \{0,1\}$	Inversion oracle
$6.(j-1)$	$\begin{cases} \xleftarrow{\mathsf{r}} \mathcal{C} \times \{0,1\}^{4\lambda} & \text{if } i \le j-1 \\ \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{*v}) & \text{if } i \ge j \end{cases}$	$\mathsf{Inv}(\mathsf{td}, \cdot)$ $^{(\dagger)}$
$6.j.a$	$\begin{cases} \xleftarrow{\mathsf{r}} \mathcal{C} \times \{0,1\}^{4\lambda} & \text{if } i \le j-1 \\ \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{*v}) & \text{if } i \ge j \end{cases}$	$\mathsf{AltInv}(\mathsf{td}, \mathsf{sk}_0, j, \mathsf{h}^*, \cdot)$
$6.j.b$	$\begin{cases} \xleftarrow{\mathsf{r}} \mathcal{C} \times \{0,1\}^{4\lambda} & \text{if } i \le j \\ \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{*v}) & \text{if } i \ge j+1 \end{cases}$	$\mathsf{AltInv}(\mathsf{td}, \mathsf{sk}_0, j, \mathsf{h}^*, \cdot)$
$6.j$	$\begin{cases} \xleftarrow{\mathsf{r}} \mathcal{C} \times \{0,1\}^{4\lambda} & \text{if } i \le j \\ \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}_i^{*v}) & \text{if } i \ge j+1 \end{cases}$	$\mathsf{Inv}(\mathsf{td}, \cdot)$ $^{(\dagger)}$

We will consider transitions of the games in the following order: $6.0 \to 6.1.a \to 6.1.b \to 6.1 \to 6.2.a \to \cdots \to 6.n.b \to 6.n$. See Table 2 for an overview of the sequence.

In these intermediate games, for $\alpha \in [n]$, we call an inversion query $\mathsf{y} = (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$ α-*bad* if it satisfies $\mathsf{H}(\mathsf{hk}, \mathsf{ct}_0 \| (\mathsf{ct}_i)_{i\in[n]}) \neq \mathsf{h}^*$ and $\mathsf{Inv}(\mathsf{td}, \mathsf{y}) \neq \mathsf{AltInv}(\mathsf{td}, \mathsf{sk}_0, \alpha, \mathsf{h}^*, \mathsf{y})$ simultaneously. For Game t and $\alpha \in [n]$, we denote by $\mathtt{BQ}_t^\alpha$ the event that $\mathcal{A}$ submits an α-bad inversion query in Game t. Also, we continue to denote by $\mathtt{T}_t$ the event that $\mathcal{A}$ outputs $b' = 1$ in Game t.

Observe that for $j \in [n]$, Game $6.(j-1)$ and Game $6.j.a$ proceed identically unless $\mathcal{A}$ submits a j-bad inversion query. Thus, we have $|\Pr[\mathtt{T}_{6.(j-1)}] - \Pr[\mathtt{T}_{6.j.a}]| \le \Pr[\mathtt{BQ}^j_{6.(j-1)}]$. Similarly, Game $6.j.b$ and Game $6.j$ proceed identically unless $\mathcal{A}$ submits a j-bad inversion query. Thus, we have $|\Pr[\mathtt{T}_{6.j.b}] - \Pr[\mathtt{T}_{6.j}]| \le \Pr[\mathtt{BQ}^j_{6.j}]$.

Using them, we can estimate $|\Pr[\mathtt{T}_6] - \Pr[\mathtt{T}_7]|$ as follows.

$$
\begin{aligned}
&\Big|\Pr[\mathtt{T}_6] - \Pr[\mathtt{T}_7]\Big| = \Big|\Pr[\mathtt{T}_{6.0}] - \Pr[\mathtt{T}_{6.n}]\Big| = \Big|\sum_{j\in[n]} \Big(\Pr[\mathtt{T}_{6.(j-1)}] - \Pr[\mathtt{T}_{6.j}]\Big)\Big| \\
&= \Big|\sum_{j\in[n]} \Big(\Pr[\mathtt{T}_{6.(j-1)}] - \Pr[\mathtt{T}_{6.j.a}] + \Pr[\mathtt{T}_{6.j.a}] - \Pr[\mathtt{T}_{6.j.b}] + \Pr[\mathtt{T}_{6.j.b}] - \Pr[\mathtt{T}_{6.j}]\Big)\Big| \\
&\le \sum_{j\in[n]} \Big|\Pr[\mathtt{T}_{6.(j-1)}] - \Pr[\mathtt{T}_{6.j.a}]\Big| + \sum_{j\in[n]} \Big|\Pr[\mathtt{T}_{6.j.b}] - \Pr[\mathtt{T}_{6.j}]\Big| \\
&\quad + \Big|\sum_{j\in[n]} \Big(\Pr[\mathtt{T}_{6.j.a}] - \Pr[\mathtt{T}_{6.j.b}]\Big)\Big| \\
&\le \sum_{j\in[n]} \Pr[\mathtt{BQ}^j_{6.(j-1)}] + \sum_{j\in[n]} \Pr[\mathtt{BQ}^j_{6.j}] + \Big|\sum_{j\in[n]} \Big(\Pr[\mathtt{T}_{6.j.a}] - \Pr[\mathtt{T}_{6.j.b}]\Big)\Big|
\end{aligned}
$$

We will show the following claims which, combined with the above inequality, imply the lemma.

Claim 3 *For all $j \in \{0\} \cup [n]$ and $\alpha \in [n]$, we have $\Pr[\mathtt{BQ}^\alpha_{6.j}] \le (n+1) \cdot \epsilon + n \cdot 2^{-\lambda}$.*

This claim is proved in exactly the same way as Claim 1, and thus its proof is omitted.

Claim 4 *There exists a PPT adversary $\mathcal{B}''$ such that*

$$\left|\sum_{j\in[n]}\left(\Pr[\mathtt{T}_{6.j.a}]-\Pr[\mathtt{T}_{6.j.b}]\right)\right| = 2n\cdot \mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM},\mathcal{B}''}(\lambda).$$

Proof of Claim 4. The proof of this claim proceed similarly to the proof of Claim 2, with the difference in how the reduction algorithm embeds its challenge instance.

Using $\mathcal{A}$ as a building block, we show how to construct a PPT adversary $\mathcal{B}''$ that attacks the pseudorandom ciphertext property of KEM with the claimed advantage. The description of $\mathcal{B}''$ is as follows.

$\mathcal{B}''(\mathsf{pk}', \mathsf{ct}'^*_\beta, \mathsf{k}'^*_\beta)$**:** (where $\beta \in \{0,1\}$ denotes $\mathcal{B}''$'s challenge bit) First, $\mathcal{B}''$ picks $u \xleftarrow{\mathsf{r}} [n]$ and $\sigma \xleftarrow{\mathsf{r}} \{0,1\}$ uniformly at random, and prepares ek and y^* as follows.

1. Pick $\mathsf{s}^* = (\mathsf{s}^*_1, \ldots, \mathsf{s}^*_n) \xleftarrow{\mathsf{r}} \{0,1\}^n$ and $\mathsf{r}^{*v}_i \xleftarrow{\mathsf{r}} \{0,1\}^{\ell_r}$ for every $(i,v) \in [n]\times\{0,1\} \setminus \{(u,\sigma)\}$. Also, pick $\mathsf{B} \xleftarrow{\mathsf{r}} \{0,1\}^{4\lambda}$.
2. Set $(\mathsf{pk}_u, \mathsf{sk}_u) \leftarrow (\mathsf{pk}', \bot)$, and compute $(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KKG}(1^\lambda)$ for every $i \in \{0,\ldots,n\}\setminus\{u\}$.
3. For every $i \in [n]$, compute/set $(\mathsf{ct}^{*0}_i, \mathsf{k}^{*0}_i)$ and $(\mathsf{ct}^{*1}_i, \mathsf{k}^{*1}_i)$ as follows:
   - If $i \le u-1$: $(\mathsf{ct}^{*v}_i, \mathsf{k}^{*v}_i) \xleftarrow{\mathsf{r}} \mathcal{C}\times\{0,1\}^{4\lambda}$ for both $v \in \{0,1\}$.
   - If $i = u$:
     - If $\sigma = 0$: $(\mathsf{ct}^{*0}_u, \mathsf{k}^{*0}_u) \leftarrow (\mathsf{ct}'^*_\beta, \mathsf{k}'^*_\beta)$ ($\mathcal{B}''$'s challenge instance) and $(\mathsf{ct}^{*1}_u, \mathsf{k}^{*1}_u) \leftarrow \mathsf{Encap}(\mathsf{pk}_u; \mathsf{r}^{*1}_u)$.
     - If $\sigma = 1$: $(\mathsf{ct}^{*0}_u, \mathsf{k}^{*0}_u) \xleftarrow{\mathsf{r}} \mathcal{C}\times\{0,1\}^{4\lambda}$ and $(\mathsf{ct}^{*1}_u, \mathsf{k}^{*1}_u) \leftarrow (\mathsf{ct}'^*_\beta, \mathsf{k}'^*_\beta)$ ($\mathcal{B}''$'s challenge instance).
   - If $i \ge u+1$: $(\mathsf{ct}^{*v}_i, \mathsf{k}^{*v}_i) \leftarrow \mathsf{Encap}(\mathsf{pk}_i; \mathsf{r}^{*v}_i)$ for both $v \in \{0,1\}$.
4. Set $\mathsf{ct}^*_i \leftarrow \mathsf{ct}^{*0}_i$ and $\mathsf{C}_i \leftarrow \mathsf{ct}^{*0}_i - \mathsf{ct}^{*1}_i$ for every $i \in [n]$.
5. Pick $\mathsf{ct}^*_0 \xleftarrow{\mathsf{r}} \mathcal{C}$ uniformly at random.
6. Compute $\mathsf{hk} \leftarrow \mathsf{HKG}(1^\lambda)$ and $\mathsf{h}^* \leftarrow \mathsf{H}(\mathsf{hk}, \mathsf{ct}^*_0 \| (\mathsf{ct}^*_i)_{i\in[n]})$.
7. Compute $\mathsf{A}_i \leftarrow \mathsf{k}^{*0}_i - \mathsf{k}^{*1}_i - \mathsf{B}\cdot\mathsf{h}^*$ and $\mathsf{T}^*_i \leftarrow \mathsf{k}^{*0}_i$ for every $i \in [n]$.
8. Set $\mathsf{ek} \leftarrow ((\mathsf{pk}_i, \mathsf{A}_i, \mathsf{C}_i)_{i\in[n]}, \mathsf{pk}_0, \mathsf{B}, \mathsf{hk})$, $\mathsf{td} \leftarrow ((\mathsf{sk}_i)_{i\in[n]}, \mathsf{ek})$, and $\mathsf{y}^* \leftarrow (\mathsf{ct}^*_0, (\mathsf{ct}^*_i, \mathsf{T}^*_i)_{i\in[n]})$.

Then, $\mathcal{B}''$ runs $\mathcal{A}(\mathsf{ek}, \mathsf{y}^*)$. From here on, $\mathcal{A}$ may start making inversion queries $\mathsf{y} = (\mathsf{ct}_0, (\mathsf{ct}_i, \mathsf{T}_i)_{i\in[n]})$, which are answered with $\mathsf{x} \leftarrow \mathsf{AltInv}(\mathsf{td}, \mathsf{sk}_0, u, \mathsf{h}^*, \mathsf{y})$. (Note that to execute AltInv here, sk_u corresponding to pk_u is not needed.)

When $\mathcal{A}$ terminates with output b', $\mathcal{B}''$ sets $\beta' \leftarrow b'$, and terminates with output β'.

It is not hard to see that if $\mathcal{B}''$'s challenge bit β is 1 and $\sigma = 0$ holds, then $\mathcal{B}''$ perfectly simulates Game $6.u.a$ for $\mathcal{A}$. Under this situation, for every $j \in [n]$, if $u = j$, then the probability that $\mathsf{Eval}(\mathsf{ek}, \mathsf{x}') = \mathsf{y}^*$ holds (and thus $\mathcal{B}''$ outputs $\beta' = 1$) is exactly $\Pr[\mathtt{T}_{6.j.a}]$, i.e. we have $\Pr[\beta' = 1 | \beta = 1 \wedge u = j \wedge \sigma = 0] = \Pr[\mathtt{T}_{6.j.a}]$. On the other hand, if $\mathcal{B}''$'s challenge bit β is 0 and $\sigma = 1$ holds, then $\mathcal{B}''$ perfectly simulates Game $6.u.b$ for $\mathcal{A}$. Thus, we have $\Pr[\beta' = 1 | \beta = 0 \wedge u = j \wedge \sigma = 1] = \Pr[\mathtt{T}_{6.j.b}]$. Since $u \in [n]$ and $\sigma \in \{0,1\}$ are chosen uniformly at random, independently of β, we have $\Pr[u = j \wedge \sigma = v | \beta = 0] = \Pr[u = j \wedge \sigma = v | \beta = 1] = 1/(2n)$ for

every $j \in [n]$ and $v \in \{0,1\}$. Hence, we have

$$\begin{aligned}
&\Pr[\beta' = 1 | \beta = 1] \\
&= \frac{1}{2n} \sum_{j \in [n]} \Big(\Pr[\beta' = 1 | \beta = 1 \wedge u = j \wedge \sigma = 0] + \Pr[\beta' = 1 | \beta = 1 \wedge u = j \wedge \sigma = 1] \Big) \\
&= \frac{1}{2n} \sum_{j \in [n]} \Big(\Pr[\mathtt{T}_{6.j.a}] + \Pr[\beta' = 1 | \beta = 1 \wedge u = j \wedge \sigma = 1] \Big).
\end{aligned}$$

Similarly, we have

$$\Pr[\beta' = 1 | \beta = 0] = \frac{1}{2n} \sum_{j \in [n]} \Big(\Pr[\mathtt{T}_{6.j.b}] + \Pr[\beta' = 1 | \beta = 0 \wedge u = j \wedge \sigma = 0] \Big).$$

Furthermore, from $\mathcal{A}$'s view point, the experiment simulated by $\mathcal{B}''$ in case $\beta = 1$ and $\sigma = 1$, and that in case $\beta = 0$ and $\sigma = 0$, are identical. Thus, for every $j \in [n]$, we have

$$\Pr[\beta' = 1 | \beta = 1 \wedge u = j \wedge \sigma = 1] = \Pr[\beta' = 1 | \beta = 0 \wedge u = j \wedge \sigma = 0].$$

Hence, $\mathcal{B}''$'s advantage can be calculated as follows:

$$\mathsf{Adv}^{\mathsf{prct}}_{\mathsf{KEM},\mathcal{B}''}(\lambda) = \Big| \Pr[\beta' = 1 | \beta = 1] - \Pr[\beta' = 1 | \beta = 0] \Big| = \frac{1}{2n} \Big| \sum_{j \in [n]} \Big(\Pr[\mathtt{T}_{6.j.a}] - \Pr[\mathtt{T}_{6.j.b}] \Big) \Big|,$$

which is equivalent to the claimed equality. □ (**Claim 4**)

□ (**Lemma 7**)

Lemma 8 *There exists a PPT adversary $\mathcal{B}'_{\mathsf{h}}$ satisfying*

$$\Big| \Pr[\mathtt{T}_6] - \Pr[\mathtt{T}_7] \Big| \leq \mathsf{Adv}^{\mathsf{tcr}}_{\mathsf{Hash},\mathcal{B}'_{\mathsf{h}}}(\lambda) + 2^{-\lambda}.$$

Proof of Lemma 8. The proof for this lemma is similar to, but somewhat simpler than, the proof of Lemma 2. Specifically, we use the same notion of *hash-bad* queries, and the same events $\mathtt{HB}^k_t$ for $t \in \{7,8\}$ and $k \in [2]$, as in the proof of Lemma 2. Then, since Game 7 and Game 8 proceed identically unless $\mathcal{A}$ makes a Type-1 or Type 2 hash bad inversion query, we have

$$\Big| \Pr[\mathtt{T}_7] - \Pr[\mathtt{T}_8] \Big| \leq \Pr[\mathtt{HB}^1_8] + \Pr[\mathtt{HB}^2_8].$$

Furthermore, as in the proof of Lemma 2, we can straightforwardly show that there exists a PPT adversary $\mathcal{B}'_{\mathsf{h}}$ with $\Pr[\mathtt{HB}^1_8] = \mathsf{Adv}^{\mathsf{tcr}}_{\mathsf{Hash},\mathcal{B}'_{\mathsf{h}}}(\lambda)$. (The proof is omitted.)

Regarding $\Pr[\mathtt{HB}^2_8]$, note that in Game 8, ct^*_0 is chosen uniformly at random from $\mathcal{C}$, and we have $|\mathcal{C}| \geq 2^{\ell_r + \lambda}$. Thus, the probability that $(\mathsf{ct}^*_0, *)$ belongs to the image of $\mathsf{Encap}(\mathsf{pk}_0; \cdot)$ is at most $2^{-\lambda}$. If $(\mathsf{ct}^*_0, *)$ is not in the image of $\mathsf{Encap}(\mathsf{pk}_0; \cdot)$, then $\mathsf{Inv}(\mathsf{td}, \mathsf{y})$ for any element y that contains ct^*_0 as its first element will result in $\perp$, since the validity check by re-encapsulation (i.e. the check of $\mathsf{Encap}(\mathsf{pk}_0; \mathsf{r}_0) = (\mathsf{ct}^*_0, *)$) performed at the second last step of Inv cannot be satisfied. In other words, except with probability at most $2^{-\lambda}$, Type-2 hash bad queries do not exist in Game 8. Hence, we have $\Pr[\mathtt{HB}^2_8] \leq 2^{-\lambda}$.

Putting everything together, there exists a PPT adversary $\mathcal{B}'_{\mathsf{h}}$ with the claimed advantage, as required. □ (**Lemma 8**)

Due to Lemmas 2 to 8 and Eq. (7), we can conclude that there exist PPT adversaries $\mathcal{B}_{\mathsf{h}}$, $\mathcal{B}'_{\mathsf{h}}$, $\mathcal{B}$, $\mathcal{B}'$, and $\mathcal{B}''$ satisfying Eq. (5). □ (**Theorem 6**)

Acknowledgement This work was partially supported by JST CREST JPMJCR22M1.

References

[ACPS09] Benny Applebaum, David Cash, Chris Peikert, and Amit Sahai. Fast cryptographic primitives and circular-secure encryption based on hard learning problems. In Shai Halevi, editor, *CRYPTO 2009*, volume 5677 of *LNCS*, pages 595–618. Springer, Berlin, Heidelberg, August 2009.

[BHY09] Mihir Bellare, Dennis Hofheinz, and Scott Yilek. Possibility and impossibility results for encryption and commitment secure under selective opening. In Antoine Joux, editor, *EUROCRYPT 2009*, volume 5479 of *LNCS*, pages 1–35. Springer, Berlin, Heidelberg, April 2009.

[BL18] Sebastian Berndt and Maciej Liskiewicz. On the gold standard for security of universal steganography. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part I*, volume 10820 of *LNCS*, pages 29–60. Springer, Cham, April / May 2018.

[BRS03] John Black, Phillip Rogaway, and Thomas Shrimpton. Encryption-scheme security in the presence of key-dependent messages. In Kaisa Nyberg and Howard M. Heys, editors, *SAC 2002*, volume 2595 of *LNCS*, pages 62–75. Springer, Berlin, Heidelberg, August 2003.

[CDG+17] Chongwon Cho, Nico Döttling, Sanjam Garg, Divya Gupta, Peihan Miao, and Antigoni Polychroniadou. Laconic oblivious transfer and its applications. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part II*, volume 10402 of *LNCS*, pages 33–65. Springer, Cham, August 2017.

[DDN91] Danny Dolev, Cynthia Dwork, and Moni Naor. Non-malleable cryptography (extended abstract). In *23rd ACM STOC*, pages 542–552. ACM Press, May 1991.

[DG17] Nico Döttling and Sanjam Garg. Identity-based encryption from the Diffie-Hellman assumption. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 537–569. Springer, Cham, August 2017.

[DH76] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.

[DNR04] Cynthia Dwork, Moni Naor, and Omer Reingold. Immunizing encryption schemes from decryption errors. In Christian Cachin and Jan Camenisch, editors, *EUROCRYPT 2004*, volume 3027 of *LNCS*, pages 342–360. Springer, Berlin, Heidelberg, May 2004.

[FHKW10] Serge Fehr, Dennis Hofheinz, Eike Kiltz, and Hoeteck Wee. Encryption schemes secure against chosen-ciphertext selective opening attacks. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 381–402. Springer, Berlin, Heidelberg, May / June 2010.

[FOR12] Benjamin Fuller, Adam O'Neill, and Leonid Reyzin. A unified approach to deterministic encryption: New constructions and a connection to computational entropy. In Ronald Cramer, editor, *TCC 2012*, volume 7194 of *LNCS*, pages 582–599. Springer, Berlin, Heidelberg, March 2012.

[GGH19] Sanjam Garg, Romain Gay, and Mohammad Hajiabadi. New techniques for efficient trapdoor functions and applications. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part III*, volume 11478 of *LNCS*, pages 33–63. Springer, Cham, May 2019.

[GH18] Sanjam Garg and Mohammad Hajiabadi. Trapdoor functions from the computational Diffie-Hellman assumption. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 362–391. Springer, Cham, August 2018.

[GHMO21] Sanjam Garg, Mohammad Hajiabadi, Giulio Malavolta, and Rafail Ostrovsky. How to build a trapdoor function from an encryption scheme. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part III*, volume 13092 of *LNCS*, pages 220–249. Springer, Cham, December 2021.

[GL89] Oded Goldreich and Leonid A. Levin. A hard-core predicate for all one-way functions. In *21st ACM STOC*, pages 25–32. ACM Press, May 1989.

[GLN11] Oded Goldreich, Leonid A. Levin, and Noam Nisan. On constructing 1-1 one-way functions. In Oded Goldreich, editor, *Studies in Complexity and Cryptography. Miscellanea on the Interplay between Randomness and Computation - In Collaboration with Lidor Avigad, Mihir Bellare, Zvika Brakerski, Shafi Goldwasser, Shai Halevi, Tali Kaufman, Leonid Levin, Noam Nisan, Dana Ron, Madhu Sudan, Luca Trevisan, Salil Vadhan, Avi Wigderson, David Zuckerman*, volume 6650 of *Lecture Notes in Computer Science*, pages 13–25. Springer, 2011.

[Gol01] Oded Goldreich. *Foundations of Cryptography: Basic Tools*, volume 1. Cambridge University Press, Cambridge, UK, 2001.

[HILL99] Johan Håstad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. *SIAM Journal on Computing*, 28(4):1364–1396, 1999.

[HKW20] Susan Hohenberger, Venkata Koppula, and Brent Waters. Chosen ciphertext security from injective trapdoor functions. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part I*, volume 12170 of *LNCS*, pages 836–866. Springer, Cham, August 2020.

[Hop05] Nicholas Hopper. On steganographic chosen covertext security. In Luís Caires, Giuseppe F. Italiano, Luís Monteiro, Catuscia Palamidessi, and Moti Yung, editors, *ICALP 2005*, volume 3580 of *LNCS*, pages 311–323. Springer, Berlin, Heidelberg, July 2005.

[KMO10] Eike Kiltz, Payman Mohassel, and Adam O'Neill. Adaptive trapdoor functions and chosen-ciphertext security. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 673–692. Springer, Berlin, Heidelberg, May / June 2010.

[KMT19] Fuyuki Kitagawa, Takahiro Matsuda, and Keisuke Tanaka. CCA security and trapdoor functions via key-dependent-message security. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 33–64. Springer, Cham, August 2019.

[KMT22] Fuyuki Kitagawa, Takahiro Matsuda, and Keisuke Tanaka. CCA security and trapdoor functions via key-dependent-message security. *Journal of Cryptology*, 35(2):9, April 2022.

[KW19] Venkata Koppula and Brent Waters. Realizing chosen ciphertext security generically in attribute-based encryption and predicate encryption. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 671–700. Springer, Cham, August 2019.

[LP15] Shengli Liu and Kenneth G. Paterson. Simulation-based selective opening CCA security for PKE from key encapsulation mechanisms. In Jonathan Katz, editor, *PKC 2015*, volume 9020 of *LNCS*, pages 3–26. Springer, Berlin, Heidelberg, March / April 2015.

[MH15] Takahiro Matsuda and Goichiro Hanaoka. Constructing and understanding chosen ciphertext security via puncturable key encapsulation mechanisms. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015, Part I*, volume 9014 of *LNCS*, pages 561–590. Springer, Berlin, Heidelberg, March 2015.

[NY89] Moni Naor and Moti Yung. Universal one-way hash functions and their cryptographic applications. In *21st ACM STOC*, pages 33–43. ACM Press, May 1989.

[NY90] Moni Naor and Moti Yung. Public-key cryptosystems provably secure against chosen ciphertext attacks. In *22nd ACM STOC*, pages 427–437. ACM Press, May 1990.

[PW08] Chris Peikert and Brent Waters. Lossy trapdoor functions and their applications. In Richard E. Ladner and Cynthia Dwork, editors, *40th ACM STOC*, pages 187–196. ACM Press, May 2008.

[QWW21] Willy Quach, Brent Waters, and Daniel Wichs. Targeted lossy functions and applications. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 424–453, Virtual Event, August 2021. Springer, Cham.

[QWZ18] Willy Quach, Daniel Wichs, and Giorgos Zirdelis. Watermarking PRFs under standard assumptions: Public marking and security with extraction queries. In Amos Beimel and Stefan Dziembowski, editors, *TCC 2018, Part II*, volume 11240 of *LNCS*, pages 669–698. Springer, Cham, November 2018.

[Rom90] John Rompel. One-way functions are necessary and sufficient for secure signatures. In *22nd ACM STOC*, pages 387–394. ACM Press, May 1990.

[RS92] Charles Rackoff and Daniel R. Simon. Non-interactive zero-knowledge proof of knowledge and chosen ciphertext attack. In Joan Feigenbaum, editor, *CRYPTO'91*, volume 576 of *LNCS*, pages 433–444. Springer, Berlin, Heidelberg, August 1992.

[RS09] Alon Rosen and Gil Segev. Chosen-ciphertext security via correlated products. In Omer Reingold, editor, *TCC 2009*, volume 5444 of *LNCS*, pages 419–436. Springer, Berlin, Heidelberg, March 2009.

[RSA78] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the Association for Computing Machinery*, 21(2):120–126, February 1978.